

Funciones en Python

¿Qué son las funciones?

- **Bloques de código reutilizables** que realizan tareas específicas.
- Actúan como *mini programas* dentro de un programa mayor.
- Se pueden ejecutar muchas veces sin reescribir el mismo código.

Importancia de las funciones

1. **Organización:** Dividen tareas complejas en partes más pequeñas y manejables (como capítulos de un libro).
2. **Reutilización:** Se definen una vez y se pueden llamar varias veces → ahorran tiempo y reducen errores.
3. **Abstracción:** Ocultan detalles internos de una tarea, dejando una interfaz simple (ej. enviar correos, cálculos).
4. **Colaboración:** Permiten trabajar en equipo porque son módulos independientes que se integran fácilmente.

Tipos de funciones en Python

1. **Funciones integradas** → ya vienen con Python (ej. `print()`, `len()`, `input()`).
2. **Funciones definidas por el usuario** → creadas por el programador para tareas específicas.
3. **Funciones lambda** → funciones anónimas de una sola línea, útiles para operaciones sencillas (ej. ordenar listas).

Sintaxis básica de una función

```
def nombre_funcion(parámetros):  
    """Docstring: explica qué hace la función"""  
    # Código que realiza la tarea  
    return valor # Devuelve un resultado
```

- **def**: palabra clave para definir la función.
- **nombre_funcion**: identificador descriptivo.
- **parámetros**: valores de entrada opcionales.
- **docstring**: texto explicativo de la función.
- **return**: envía un valor de salida al programa.

Pregunta

En un gran proyecto de software con múltiples desarrolladores, ¿cuál de las siguientes opciones explica mejor cómo las funciones de Python contribuyen a una colaboración eficiente? Seleccione la mejor respuesta.

- ☒ Las funciones permiten el desarrollo paralelo de distintas partes del proyecto, lo que permite a varios desarrolladores trabajar en unidades de código autónomas que posteriormente pueden integrarse sin problemas.
- ☐ Las funciones permiten a los desarrolladores trabajar en partes aisladas del código base sin necesidad de comprender la estructura de todo el proyecto.
- ☐ Las funciones proporcionan un vocabulario compartido para debatir y comprender partes específicas del código, lo que facilita una comunicación clara entre los miembros del equipo.
- ☐ Las funciones animan a los desarrolladores individuales a escribir su código con estilos únicos, fomentando la creatividad y la diversidad dentro del proyecto.

✓ Correcto

Correcto Capta la esencia de cómo las funciones fomentan el trabajo en equipo al permitir a los desarrolladores trabajar de forma independiente en piezas modulares de código, lo que en última instancia acelera el desarrollo y reduce los conflictos.

Clases en Python

Las **clases** son **planos o moldes** para crear objetos en Python.

Definen **atributos** (datos/características) y **métodos** (acciones/comportamientos), lo que permite modelar entidades del mundo real y organizar mejor el código.

Ejemplo sencillo:

- Clase **Coche** → define color, motor, asientos, y métodos como **acelerar()**, **frenar()** o **girar()**.
- Cada coche creado a partir de la clase es un **objeto**.

Habilidades clave de las clases

1. **Encapsulación:** Agrupan datos y funciones en una sola unidad, promoviendo organización y modularidad.
2. **Herencia:** Permite que una clase herede atributos y métodos de otra (ejemplo: `Vehículo` → `Camión`, `Motocicleta`).
3. **Polimorfismo:** Diferentes clases pueden responder al mismo método de forma distinta (ejemplo: `Shape` → `Circle`, `Square`, `Triangle` con su propio `draw()`).

Componentes de una clase

- **Atributos:** Variables que guardan datos del objeto (ejemplo: `color`, `balance`, `salud`).
- **Métodos:** Funciones que definen el comportamiento del objeto (ejemplo: `depositar`, `mover`, `atacar`).
- **Constructor (`__init__`):** Método especial que inicializa los atributos al crear un objeto.

Instanciación

- Proceso de crear un objeto a partir de una clase.
- Ejemplo: `BankAccount` → al instanciar, se crea `alice_account` con atributos y métodos propios.

Ejemplos prácticos de uso

- **Gestión de inventario:** Clase `Producto` con atributos (`nombre`, `precio`, `stock`) y métodos (`actualizar_stock()`, `generar_reporte()`).
- **Videojuegos:** Clase `Personaje` con atributos (`salud`, `fuerza`, `posición`) y métodos (`atacar()`, `mover()`, `recibir_daño()`).

Pregunta

¿Cuál de los siguientes escenarios demostraría el concepto de polimorfismo en las clases de Python? Seleccione la mejor respuesta.

- ☒ Una clase `Vehicle` con subclases como `Car` y `Motorcycle` que comparten los métodos de `Vehicle` y añaden los suyos propios.
- ☐ Una clase `Product` con atributos como `name` y `price`, utilizada para crear una lista de productos en un sistema de inventario.
- ☐ Una clase `BankAccount` con métodos para `deposit` y `withdraw`, donde el atributo `balance` se actualiza en función de la transacción.
- ☐ Una clase `Customer` con atributos como `name` y `email`, y un método `send_email` que envía un correo electrónico personalizado al cliente.

☒ Correcto

Correcto Ilustra el polimorfismo, ya que diferentes subclases (Coche, Motocicleta) responden al mismo mensaje (`accelerate()`) de formas distintas, reflejando sus características únicas.

El arte de la abstracción, funciones y DRY

El **principio DRY** (*Don't Repeat Yourself*) es una filosofía clave en la programación moderna que busca **evitar la repetición de código**.

En lugar de copiar y pegar bloques similares en varias partes del programa, DRY recomienda **centralizar la lógica en funciones reutilizables**.

Ventajas del principio DRY:

- Menos errores (no hay que modificar el mismo código en varios lugares).
- Más consistencia (una sola fuente de verdad).
- Código más limpio, organizado y fácil de mantener.

Funciones: bloques reutilizables de código

Las **funciones** permiten encapsular tareas específicas.

- **Argumentos (entradas):** datos que recibe la función (ejemplo: `length`, `width`).
- **Valor de retorno (salida):** el resultado que la función devuelve (ejemplo: área del rectángulo).

Esto promueve un diseño **modular**, donde el programa se construye con funciones pequeñas y claras, en lugar de un único bloque grande de código.

Ejemplo en Python: función para calcular el área de un rectángulo

```
def calculate_area(length, width):  
    """  
    Calcula el área de un rectángulo.  
    Parámetros:  
    length (float): largo del rectángulo  
    width (float): ancho del rectángulo  
    Retorna:  
    float: área del rectángulo  
    """  
    area = length * width  
    return area  
  
# Ejemplo de uso  
  
length = 5  
width = 3  
  
rectangle_area = calculate_area(length, width) # Llamada a la  
función con entradas  
  
print(f"The area of the rectangle is: {rectangle_area}") # Usando  
la salida
```

Salida:

```
The area of the rectangle is: 15
```

En conclusión:

El principio **DRY + funciones** convierte programas en sistemas más **eficientes, organizados y fáciles de ampliar**, evitando redundancias y fomentando la reutilización del código.

DRY y funciones: una combinación perfecta

- Las **funciones** son ideales para aplicar el principio **DRY (Don't Repeat Yourself)**.
- Permiten encapsular código reutilizable, evitando duplicaciones.
- Hacen el código más **limpio, organizado, comprobable y fácil de depurar**.

Ventajas de las funciones y DRY

1. **Reutilización del código** → evita "reinventar la rueda".
2. **Escalabilidad** → facilita dividir grandes problemas en piezas más pequeñas y añadir nuevas funciones sin afectar el resto.
3. **Mantenibilidad** → se pueden hacer cambios en una función sin alterar todo el sistema.
4. **Trabajo en equipo** → funciones claras permiten dividir tareas entre varios desarrolladores.
5. **Pruebas unitarias** → cada función puede probarse de manera aislada.
6. **Legibilidad** → mejora la comprensión del código al encapsular la lógica.
7. **Depuración** → facilita localizar errores en funciones específicas.

En conjunto, funciones + DRY hacen posible proyectos **más escalables, mantenibles, colaborativos y robustos**.

Refactorización

- **Definición:** proceso de **reestructurar el código existente sin cambiar su comportamiento externo**.
- **Objetivo:** mejorar **diseño, legibilidad y mantenibilidad**.
- **Ejemplo:** en una tienda online, si el cálculo del impuesto se repite en varios lugares, se refactoriza en una sola función:

```
def calculate_final_price(price, tax_rate):  
    return price + (price * tax_rate)
```

Luego, en lugar de repetir el cálculo en cada página, se llama a esta función.

El resultado para el usuario es el mismo, pero el código es más **fácil de actualizar y menos propenso a errores**.

Conclusión

El principio **DRY** junto con el uso de **funciones** es clave para un software:

- **limpio, eficiente, escalable y mantenible.**
La **refactorización** asegura que, incluso después de escribir el código, podamos **mejorarlo y adaptarlo** sin alterar su comportamiento, manteniendo el software sano a largo plazo.

El arte de equilibrar DRY y las funciones en Python

DRY con equilibrio

- **DRY (Don't Repeat Yourself)** es valioso, pero **no absoluto**.
- Aplicarlo en exceso genera sobrecarga: demasiadas funciones triviales pueden complicar la lectura y el mantenimiento.
- El objetivo es un código **claro, conciso y fácil de entender**, no solo "DRY".

Criterios para decidir si crear una función:

1. **Frecuencia de uso** → si se repite, conviene encapsularlo.
2. **Complejidad** → si la lógica es extensa, mejora la legibilidad.
3. **Probabilidad de cambio** → si es probable que evolucione, centralizarlo en una función simplifica las modificaciones.

Funciones en el ecosistema de Python

Python potencia las funciones con herramientas modernas:

- **Decoradores** → modifican el comportamiento de funciones (ej. `@cache`).
- **Generadores** → producen datos bajo demanda, eficientes en memoria.
- **F-strings** → formateo legible y dinámico de cadenas.
- **Type hints** → anotaciones de tipos que aclaran entradas y salidas, ayudando a detectar errores antes.

Rol en el desarrollo moderno

- Las funciones permiten **modularizar procesos complejos**, como:
 - Limpieza y preprocesamiento de datos.
 - Extracción de características.
 - Entrenamiento de modelos de machine learning.
- También encapsulan interacciones con **bases de datos, APIs y servicios en la nube**, lo que aísla el código de cambios externos.

Conclusión

- DRY y las funciones no son solo estilo: son claves para escribir código **mantenible, escalable y elegante**.
- El verdadero arte está en **equilibrar reutilización y simplicidad**.
- Adoptar estas prácticas prepara al desarrollador para proyectos complejos, trabajo en equipo y desarrollo profesional sólido en Python.

Funciones integradas en Python

¿Qué son?

- Bloques de código **preempaquetados** que realizan tareas comunes sin necesidad de reescribir lógica.
- Vienen incluidas en Python y se usan directamente.

Beneficios principales

1. **Eficiencia** → eliminan repeticiones y ahorran tiempo.
2. **Legibilidad** → el código es más claro y autodocumentado.
3. **Fiabilidad** → están probadas, optimizadas y son seguras de usar.

Funciones integradas destacadas

- **print()** → muestra resultados en la consola.
 - Ejemplo: registrar interacciones en un sistema de soporte.
- **len()** → devuelve la cantidad de elementos de una secuencia (lista, cadena, tupla).
 - Ejemplo: contar productos en un carrito de compras o validar longitud de contraseñas.
- **input()** → solicita datos al usuario y los devuelve como texto.

- Ejemplo: pedir nombre, dirección o preferencias en una aplicación.
- **type()** → devuelve el tipo de un valor o variable.
 - Ejemplo: verificar formato de datos antes de cálculos en una simulación científica.
- **int(), float(), str()** → convierten datos entre tipos:
 - **int()** → convierte a entero.
 - **float()** → convierte a número decimal.
 - **str()** → convierte a cadena de texto.

Actividades de práctica sugeridas

1. Crear una **calculadora sencilla** con **input()**, operadores y **print()**.
2. Escribir un programa que use **len()** y **type()** para **validar contraseñas u otros datos**.
3. Pedir datos al usuario y convertirlos con **int()**, **float()** y **str()**, luego realizar operaciones con cada uno.

Conclusión:

Las funciones integradas son herramientas básicas y poderosas en Python que aumentan la eficiencia, la claridad y la fiabilidad del código. Practicarlas es clave para dominar su uso.

Pregunta

Estás trabajando en un script de Python que necesita procesar la entrada del usuario, realizar cálculos y presentar los resultados de forma clara y organizada. ¿Qué combinación de funciones integradas sería la más adecuada para lograr este objetivo? Selecciona la mejor respuesta.

- ☐ `abs()`, `round()`, y `sorted()`
- ☐ `len()`, `type()`, y `int()`, `float()`, `str()`
- ☐ `open()`, `read()`, y `write()`
- ☒ `input()` operadores aritméticos (+, -, *, /), y `print()`

✓ Correcto

Correcto En esta respuesta, `input()` recoge los datos del usuario, los operadores aritméticos se encargan de los cálculos y `print()` muestra los resultados organizados, cumpliendo los requisitos del script.

ACTIVIDAD de funciones integradas

Temperaturas árticas: Diversión con las Funciones matemáticas integradas en Python

Ejercicio Matemático 1: Análisis de las Temperaturas Antárticas

Instrucciones:

1. Comience con la lista `antarctic_temperatures`, que contiene los registros diarios de temperatura.
2. Aplique la función `max()` a la lista `antarctic_temperatures` para obtener la temperatura más alta registrada.
3. Aplique la función `min()` a la lista `antarctic_temperatures` para determinar la temperatura más baja registrada.
4. Imprima las temperaturas más alta y más baja.

5. Calcule la temperatura media sumando todas las temperaturas de la lista `antarctic_temperatures` mediante la función `sum()` y dividiendo por el número de temperaturas, que puede determinar mediante la función `len()`. Redondee la temperatura media calculada a **un decimal** utilizando la función `round()`.
6. Imprima la temperatura media redondeada.
7. Calcule el valor absoluto de la temperatura más fría utilizando la función `abs()`.
8. Imprime el valor absoluto de la temperatura más fría.

```
antarctic_temperatures = [-25.5, -28.0, -26.3, -23.8, -27.1, -24.9, -  
29.2]
```

```
# Find the highest and lowest temperatures
```

```
highest_temp = max(antarctic_temperatures)
```

```
lowest_temp = min(antarctic_temperatures)
```

```
print("Highest temperature:", highest_temp, "°C")
```

```
print("Lowest temperature:", lowest_temp, "°C")
```

```
# Calculate the average temperature

average_temp = sum(antarctic_temperatures)/len(antarctic_temperatures)

print("Average temperature:", average_temp, "°C")


# Find the absolute value of the coldest temperature

coldest_temp_abs = abs(lowest_temp)

print("The coldest temperature was", coldest_temp_abs, "°C below
freezing.")
```

Restablecer

```
Highest temperature: -23.8 °C
Lowest temperature: -29.2 °C
Average temperature: -26.4 °C
The coldest temperature was 29.2 °C below freezing.
```

Conversiones de texto: Funciones integradas de conversión en Python

Instrucciones:

1. Comience con las variables proporcionadas `int_value`, `float_value`, y `text_value`.
2. Utilice la función `type()` para guardar el tipo de datos de `float_value` en la variable `type_of_float_value` donde dice **STEP 2: YOUR CODE HERE**. La función `type` se puede utilizar para determinar el tipo de datos de las variables, y se puede utilizar para evitar errores y excepciones.
3. Utilice la función `int()` para convertir el valor de `text_value` a un número entero y almacenarlo en la variable `text_value_as_int` donde dice **STEP 3: YOUR CODE HERE**. Incluso si una cadena contiene un valor numérico, debe ser convertida a un valor numérico (`int` o `float`) para ser utilizada como parte de operaciones matemáticas.
4. Utilice la función `str()` para convertir el valor de `int_value` en una cadena y almacenarlo en la variable `int_value_as_text` donde dice **STEP 4: YOUR CODE HERE**. Muchas veces, los valores numéricos que no tendrán operaciones matemáticas realizadas sobre ellos (por ejemplo, números de tarjetas de crédito, números de serie, códigos postales americanos) serán convertidos a texto.
5. Los resultados se imprimirán. Verás el tipo de datos de la variable `float`, seguido de los resultados de sumar dos valores numéricos, seguido de los resultados de sumar esos mismos dos valores como cadenas.

```
int_value = 15
```

```
float_value = 4.1
```

```
text_value = "33"
```

```
type_of_float_value = type(float_value)
```

```
# Convert text_value to an integer
```

```
text_value_as_int = int(text_value)
```

```
# Convert int_value to text
```

```
int_value_as_text = str(int_value)
```

```
# DO NOT CHANGE LINES BELOW
```

```
# Print the type of float_value
```

```
print("float_value type:", type_of_float_value)
```

```
# Adding text_value_as_int to int_value

print("Integer addition: Adding text_value_as_int (33) to int_value
(15):", text_value_as_int + int_value)


# Adding (concatenating) text values

print("Text addition: Adding text_value (33) to int_value_as_text
(15):", text_value + int_value_as_text)
```

Restablecer

```
float_value type: <class 'float'>
Integer addition: Adding text_value_as_int (33) to int_value (15): 48
Text addition: Adding text_value (33) to int_value_as_text (15): 3315
```

Las funciones integradas de Python proporcionan un conjunto de herramientas versátiles para manejar varios tipos de datos y tareas. Te permiten escribir código conciso y eficiente sin depender de librerías externas. Si dominas estas funciones, mejorarás tu productividad y desbloquearás todo el potencial de Python para el análisis de datos, el procesamiento de texto y mucho más.

Algunas funciones integradas adicionales que vale la pena explorar incluyen:

`float()` CONVERTIR UN VALUE EN UN VALOR DE PUNTO FLOTANTE

`bool()` convertir un valor en booleano

Además, habrá bibliotecas externas especializadas. Más allá de las funciones, varios tipos de datos en Python también vienen equipados con sus propias herramientas especializadas llamadas métodos. Estos métodos, a los que se accede usando un punto (.) después del objeto, te permiten realizar acciones específicas de ese tipo - por ejemplo, manipular cadenas (como `.upper()` o `.lower()`).

Módulos

Los **módulos en Python** son contenedores de código que agrupan funciones, clases y variables relacionadas, permitiendo organizar proyectos grandes de manera más clara y manejable. En lugar de tener un único archivo gigante, se dividen los programas en **módulos más pequeños**, cada uno enfocado en una tarea específica, similar a tener compartimentos en una caja de herramientas. Esto hace que el código sea más **modular, legible y fácil de mantener o probar**.

Para usar el código de un módulo, se emplea la sentencia `import`, que permite acceder a las funciones y recursos de ese módulo. Python incluye módulos integrados (como `math` para funciones matemáticas) y también permite importar módulos creados por otros desarrolladores, lo que **ahorra tiempo y evita escribir código desde cero**.

En resumen, los módulos **organizan el código, facilitan su mantenimiento y aumentan la productividad del desarrollador**.

Cuestionario funciones, clases y módulos

1. Se te encarga desarrollar un programa para calcular el área de diferentes formas geométricas. Decides utilizar funciones para esta tarea. ¿Cómo beneficia específicamente a este programa el uso de funciones? Selecciona la mejor respuesta.

- ☐ Las funciones mostrarán el área calculada en un formato fácil de usar, como una representación gráfica de la forma con su área etiquetada.
- ☒ Las funciones le permitirán calcular el área de varias formas sin necesidad de reescribir las fórmulas varias veces, reduciendo la duplicación de código.
- ☐ Las funciones detectarán automáticamente el tipo de forma y calcularán el área en consecuencia, eliminando la necesidad de introducir datos por parte del usuario.
- ☐ Las funciones harán que el programa se ejecute más rápido, ya que las llamadas a funciones son intrínsecamente más rápidas que ejecutar el código directamente dentro del programa.

✔ **Correcto**

¡Correcto! Las funciones encapsulan la lógica de cálculo del área para cada forma, permitiéndole reutilizarlas para diferentes formas sin reescribir el código.

2. Estás diseñando un juego en el que los jugadores pueden elegir diferentes personajes con habilidades únicas. ¿Cómo utilizarías las clases para representar a estos personajes? Selecciona todas las que corresponda.

- ☒ Implementa el polimorfismo haciendo que diferentes clases de caracteres respondan de forma única a la misma llamada de método, como `use_ability()`.

✔ **Correcto**

Correcto Esto permite a los personajes realizar acciones a su manera, lo que añade diversidad a la jugabilidad.

- ☒ Cree una clase genérica `Character` con atributos comunes como `health` y `name`, y métodos como `move` y `attack`.

✔ **Correcto**

Correcto Esto establece una clase base para los rasgos de carácter compartidos.

- ☒ Defina clases distintas para cada tipo de carácter (por ejemplo, `Warrior`, `Mage`, `Archer`) que hereden de la clase `Character`.

✔ **Correcto**

Correcto Esto permite a los personajes especializados heredar rasgos básicos y añadir otros únicos.

- ☐ Utiliza la clase `Character` para crear directamente objetos de personaje individuales con nombres y habilidades específicos.

3. Un equipo de desarrolladores está creando una aplicación de planificación financiera. Necesitan crear una función que calcule el interés compuesto. ¿Cuál es la mejor manera de aplicar el principio DRY y el concepto de funciones para implementar esta función? Seleccione la mejor respuesta.

- ☐ Copie y pegue la fórmula de cálculo del interés compuesto donde sea necesario en el código de la aplicación.
- ☒ Cree una única función llamada `calculate_compound_interest` que tome los parámetros necesarios (principal, tipo de interés, periodo de tiempo, etc.) y devuelva el interés compuesto calculado.
- ☐ Calcule manualmente el interés compuesto cada vez que sea necesario dentro de la aplicación.
- ☐ Escriba un bloque de código independiente para calcular el interés compuesto para cada tipo de inversión (por ejemplo, cuentas de ahorro, bonos, acciones) dentro del flujo principal del programa.

☒ **Correcto**

Correcto Este enfoque se adhiere al principio DRY encapsulando la lógica de cálculo del interés compuesto en una función reutilizable.

4. Usted está desarrollando un programa que requiere la interacción del usuario para recopilar información y luego utiliza esa información para realizar cálculos. ¿Qué Función integrada sería esencial para recoger la información del usuario? Seleccione la mejor respuesta.

- ☐ `print()`
- ☐ `len()`
- ☒ `input()`
- ☐ `type()`

☒ **Correcto**

Correcto: `input()` es la función integrada diseñada específicamente para pedir al usuario que introduzca datos y devolver su respuesta en forma de cadena.

5. Un Equipo de desarrollo está trabajando en un gran proyecto de Python para analizar datos meteorológicos. Han creado varias funciones para tareas como leer datos de diferentes fuentes, limpiar y procesar los datos, realizar análisis estadísticos y visualizar los resultados. ¿Cuál es la mejor forma de organizar estas funciones mediante módulos para mejorar la estructura y la capacidad de mantenimiento del código? Selecciona la mejor respuesta.

- ☐ Distribuir aleatoriamente las funciones entre varios módulos sin una organización clara.
- ☐ Evite por completo el uso de módulos, ya que añaden una complejidad innecesaria al proyecto.
- ☐ Coloca todas las funciones en un único archivo Python de gran tamaño.
- ☒ Agrupe funciones relacionadas en módulos independientes en función de su funcionalidad (por ejemplo, un `data_reader module`, un `data_processing module`, etc.).

✓ **Correcto**

Correcto Este enfoque fomenta la modularidad, lo que facilita la comprensión, el mantenimiento y la reutilización del código.

Escribir tus propias funciones

Las **funciones en Python** son bloques de código independientes que realizan tareas específicas, permitiendo **reutilizar código** y organizar programas de manera más estructurada. Puedes crear tus propias funciones para adaptarlas a necesidades particulares de un proyecto.

Componentes principales:

- **Argumentos:** Son los datos que se pasan a la función para que realice su tarea. Ejemplo: salario bruto, tasa impositiva y deducciones para calcular el salario neto.
- **Valores devueltos:** Son los resultados que la función devuelve al programa principal después de procesar los argumentos. Ejemplo: una función que verifica si una contraseña es segura puede devolver `True` o `False`.

Pasos para crear una función:

1. **Definir la función** con la palabra clave `def`, seguido del nombre de la función y los argumentos entre paréntesis.

2. **Agregar una cadena de documentación** (`docstring`) que explique el propósito, los argumentos y el valor devuelto.
3. **Escribir el código** dentro del bloque indentado para realizar la tarea deseada.
4. **Devolver un valor** con `return` si la función necesita producir un resultado.
5. **Guardar la función** y llamarla desde otras partes del programa para ejecutar su código.

Ventajas:

- Permiten **modularidad**, dividiendo el código en partes más manejables.
- Facilitan **reutilización**, ya que se puede llamar la misma función múltiples veces con distintos argumentos.
- Mejoran la **adaptabilidad y eficiencia**, permitiendo manejar diferentes escenarios y valores de entrada.

Ejemplo práctico: Una función que genere informes de ventas tomando como argumento el período de tiempo (semanal, mensual o anual) y devuelva el informe correspondiente.

Pregunta

Estás trabajando en un proyecto de Python y necesitas implementar un algoritmo complejo para la encriptación de datos. ¿Cuál de las siguientes opciones explica mejor la ventaja de encapsular este algoritmo dentro de una función? Selecciona la mejor respuesta.

- ☐ El nombre de la función indicará claramente su propósito, haciendo que su código sea autodocumentado.
- ☐ La función tratará automáticamente los casos de error y evitará que su programa se bloquee si se encuentran datos no válidos.
- ☒ Otros desarrolladores pueden utilizar la función para cifrar datos sin necesidad de entender los intrincados detalles del algoritmo de encriptación.
- ☐ Puedes reutilizar fácilmente la lógica de encriptación en diferentes partes de tu programa sin tener que reescribir el código cada vez.

✓ **Correcto**

Correcto. La principal ventaja de la encapsulación es que, al ocultar detalles de implementación complejos tras una interfaz sencilla, hace que el código sea más fácil de usar y mantener para los demás.

Ámbito variable

El **alcance de una variable** determina **dónde se puede acceder a ella** y **cuánto tiempo existe en el código**. Existen dos tipos principales:

1. Variables locales

- Se crean **dentro de una función** y solo son accesibles dentro de esa función.
- **Desaparecen** una vez que la función termina.
- Ayudan a **organizar el código** y a **evitar cambios accidentales** fuera de la función.

2. Variables globales

- Se crean fuera de cualquier función y son accesibles **en todo el código**.
- Útiles para **información que necesita ser compartida** entre diferentes partes del programa.
- **Precaución:** modificar variables globales dentro de funciones puede causar **efectos inesperados** en otras partes del código.

Conclusión

- Las variables locales mantienen el código **ordenado y seguro**, mientras que las globales permiten **compartir información**, pero requieren cuidado para evitar errores.
- Entender el alcance es clave para escribir código Python **limpio, eficiente y predecible**.

Ámbito de variables en Python

El **ámbito** determina **dónde se puede acceder a una variable** y **por cuánto tiempo existe**. Se puede comparar con un almacén organizado:

- **Variables globales:** Están en el “área principal del almacén” y son accesibles desde cualquier parte del código.
- **Variables locales:** Están dentro de “habitaciones” específicas (funciones o bloques) y solo son accesibles allí.

Ámbito local

- Las variables locales se crean dentro de funciones o bloques de código.
- Solo son relevantes dentro de esa función.
- Se destruyen cuando la función termina, liberando memoria.
- Permiten **encapsulación**, **evitan colisiones de nombres** y facilitan la **administración de memoria**.

Ejemplo de código

```
def calculate_average(numbers):  
    total = 0          # Variable local  
    count = len(numbers) # Variable local  
    for num in numbers:  
        total += num # total = total + num  
    average = total / count  
    return average  
  
print(calculate_average([2, 5, 4, 1]))
```

Salida:

3.0

Explicación:

- `total` y `count` son variables locales de la función `calculate_average`.
- Solo existen mientras la función se ejecuta.
- No se pueden usar fuera de la función; de intentarlo, Python dará un `NameError`.

Ámbito global

- Las **variables globales** son accesibles desde cualquier parte del módulo.
- Se declaran **fuera de funciones o bloques de código**, normalmente al inicio del archivo.
- Permiten compartir información entre varias funciones sin necesidad de pasarla como argumento.

Ejemplo de código

```
# Variable global

COMPANY_NAME = "Global Tech Solutions"


def print_welcome_message():

    print("Welcome to", COMPANY_NAME.upper())


def print_contact_info():

    print("Contact", COMPANY_NAME, "at
info@globaltechsolutions.com")


print_welcome_message()

print_contact_info()
```

Salida:

```
Welcome to GLOBAL TECH SOLUTIONS

Contact Global Tech Solutions at info@globaltechsolutions.com
```

Explicación:

- `COMPANY_NAME` es una variable global utilizada por ambas funciones.
- Representa un valor constante y accesible desde cualquier parte del módulo.

Precauciones y mejores prácticas

1. **Evitar uso excesivo:** Demasiadas variables globales dificultan el seguimiento, depuración y pruebas del código.
2. **Modificaciones cuidadosas:** Cambiar una variable global dentro de una función puede afectar otras partes del programa inesperadamente.
3. **Uso recomendado:**
 - Almacenar **constantes** o valores que no cambian.
 - Compartir datos entre varias funciones de manera controlada.
4. **Modularización:** Divide el código en módulos más pequeños para reducir la necesidad de variables globales y mejorar la mantenibilidad.

Conclusión:

Las variables globales son útiles para datos compartidos y constantes, pero deben usarse con cuidado para mantener el código **organizado, eficiente y fácil de mantener**.

Regla LEGB

Python busca el valor de una variable siguiendo un **orden jerárquico** llamado **LEGB**:

1. **Local (L):** Busca primero dentro de la función o bloque actual.
2. **Enclosing/Envuelto (E):** Busca en funciones externas si existen funciones anidadas.
3. **Global (G):** Busca en el ámbito global del módulo.
4. **Built-in (B):** Busca en los nombres predefinidos de Python, como `print()` o `list()`.

Esta jerarquía evita conflictos cuando hay variables con el mismo nombre en distintos niveles.

Ámbitos anidados (Enclosing)

- Las funciones pueden estar **anidadas** dentro de otras.
- Una variable en la función externa es accesible desde la función interna, aunque no sea global.

Ejemplo de código:

```
def outer_function():  
    outer_var = "Hello from the outer function!"  
  
    def inner_function():  
        print(outer_var) # Accesible desde el ámbito envolvente  
  
    inner_function()  
  
outer_function()
```

Salida:

```
Hello from the outer function!
```

Modificación de variables globales

- Se puede usar `global` para modificar variables globales dentro de una función:

```
counter = 0
```

```
def increment():
```

```
global counter

counter += 1

increment()

print(counter) # 1
```

- **Precaución:** Esta práctica puede causar efectos secundarios y dificultar la depuración. Mejor pasar variables como argumentos y devolver resultados.

Mejores prácticas de ámbito

1. Usa **variables locales** siempre que sea posible.
2. Limita el uso de **variables globales** a constantes o datos compartidos.
3. Elige **nombres descriptivos** para que el código sea legible.
4. Evita usar el **mismo nombre** para variables locales y globales.

Conclusión:

Entender el ámbito y la regla LEGB permite que las variables se usen de manera correcta y organizada, previniendo conflictos y facilitando la lectura, mantenimiento y depuración del código Python.

Creación de clases personalizadas en Python

Las **clases** son la base de la **Programación Orientada a Objetos (POO)**. Funcionan como **planos o moldes** para crear objetos.

- **Objetos** → entidades individuales que representan algo del mundo real o abstracto, con sus propios datos y acciones.
- **Clases** → definen los **atributos** (propiedades) y **métodos** (comportamientos) que compartirán sus objetos.

Conceptos clave de la POO

1. **Encapsulación** → Agrupar datos y métodos dentro de una clase, ocultando los detalles internos para proteger el estado de los objetos.
2. **Abstracción** → Mostrar solo lo esencial, ocultando lo complejo. Facilita entender y usar los objetos.
3. **Herencia** → Crear nuevas clases a partir de otras, reutilizando y ampliando código.
4. **Polimorfismo** → Permitir que distintos objetos respondan de forma diferente a la misma acción, aportando flexibilidad.

EJEMPLO DE CLASE PERSONALIZADA

```
class Animal:

    def __init__(self, name, species):

        self.name = name

        self.species = species

    def make_sound(self):

        return "Este animal hace un sonido."
```

```
# Herencia: León como clase hija

class Lion(Animal):

    def __init__(self, name, age, mane_color):

        super().__init__(name, "León")

        self.age = age

        self.mane_color = mane_color


    def make_sound(self):

        return "¡Rooooar!"


# Crear objetos

simba = Lion("Simba", 5, "dorado")

print(simba.name, simba.species, simba.mane_color)

print(simba.make_sound())
```

Salida:

nginx

Copiar código

Simba León dorado

¡Rooooar!

Importancia de las Clases en Python

En Python básico usamos **variables, funciones y módulos**. Pero cuando los programas crecen, estas piezas pueden desordenarse.

Las **clases** ayudan a **organizar, reutilizar y abstraer código**, además de aplicar **encapsulación**.

1. Organización del código

Las clases agrupan datos y acciones en un mismo lugar, facilitando la lectura y mantenimiento.

Ejemplo: personajes en un juego.

```
class Character:

    def __init__(self, name, health, strength, speed):

        self.name = name

        self.health = health

        self.strength = strength

        self.speed = speed


# Crear personajes

hero = Character("Héroe", 100, 20, 10)

villain = Character("Villano", 120, 25, 8)


print(hero.name, hero.health)

print(villain.name, villain.strength)
```

2. Reutilización

Una clase permite crear múltiples **objetos** sin reescribir código.

Ejemplo: cuentas de banco.

```
class BankAccount:

    def __init__(self, owner, balance=0):

        self.owner = owner

        self.balance = balance

    def deposit(self, amount):

        self.balance += amount

    def withdraw(self, amount):

        self.balance -= amount

# Crear instancias

my_account = BankAccount("Alice", 500)
your_account = BankAccount("Bob", 300)

my_account.deposit(200)

print(my_account.balance)  # 700
```

3. Encapsulación

Se usa un **guion bajo** `_` para indicar atributos privados, protegidos contra modificaciones directas.

✓ **Ejemplo:**

```
class BankAccount:

    def __init__(self, balance):
```

```
        self._balance = balance # atributo privado

    def deposit(self, amount):

        self._balance += amount

    def get_balance(self):

        return self._balance

account = BankAccount(1000)

account.deposit(500)

print(account.get_balance()) # 1500
```

(Acceder directamente con `account._balance` es posible, pero rompe la encapsulación.)

4. Abstracción

Permite centrarse en **qué hace un objeto** y no en **cómo lo hace**.
Se implementa con **clases abstractas** (módulo `abc`).

Ejemplo:

```
from abc import ABC, abstractmethod

class Animal(ABC): # Clase abstracta

    @abstractmethod

    def make_sound(self):

        pass
```



```
class Dog(Animal): # Subclase concreta

    def make_sound(self):

        return "Bark!"

dog = Dog()

print(dog.make_sound()) # Bark!
```

Resumen final

- **Organización** → agrupan datos y métodos en una misma clase.
- **Reutilización** → crear múltiples objetos sin repetir código.
- **Encapsulación** → protege atributos internos con `_`.
- **Abstracción** → define estructuras generales que deben implementarse en subclases.

Anatomía de una clase

Las clases son **moldes** para crear objetos. Agrupan **atributos** (estado) y **métodos** (comportamiento). A continuación tienes los componentes clave y ejemplos de código.

1. Definición de la clase

Se usa la palabra clave `class` y, por convención, el nombre en CamelCase.

```
class Car:

    # ... (class contents)
```

2. Atributos (datos)

Los atributos describen el estado de cada objeto. Normalmente se inicializan en el constructor `__init__`.

```
class Car:

    def __init__(self, make, model, year, color):

        self.make = make

        self.model = model

        self.year = year

        self.color = color

        self.fuel_level = 100 # Initial fuel level
```

Cada instancia de `Car` tendrá su propio `make`, `model`, `year`, `color` y `fuel_level`.

3. Métodos (comportamientos)

Los métodos son funciones dentro de la clase que manipulan atributos o realizan acciones. `self` referencia al objeto actual.

```
class Car:

    # ... (attributes)

    def start_engine(self):

        print(f"The {self.year} {self.make} {self.model}'s engine
roars to life!")

    def accelerate(self):

        print(f"The {self.color} {self.make} {self.model} picks up
speed!")
```

```
def brake(self):  
  
    print(f"The {self.make} {self.model} slows down.")
```

Observa cómo los métodos usan `self.attribute_name` para acceder a los atributos del objeto.

4. `__init__`: el constructor

`__init__` se ejecuta automáticamente al crear una instancia y garantiza que el objeto empiece con un estado válido.

```
class Car:  
  
    def __init__(self, make, model, year, color):  
  
        # Inicializa atributos  
  
        self.make = make  
  
        self.model = model  
  
        self.year = year  
  
        self.color = color  
  
        self.fuel_level = 100
```

Ejemplo completo (clase `Car` funcional)

```
class Car:  
  
    def __init__(self, make, model, year, color):  
  
        self.make = make  
  
        self.model = model  
  
        self.year = year
```

```
        self.color = color

        self.fuel_level = 100

    def start_engine(self):

        print(f"The {self.year} {self.make} {self.model}'s engine
roars to life!")

    def accelerate(self):

        if self.fuel_level > 0:

            self.fuel_level -= 5

            print(f"The {self.color} {self.make} {self.model} picks
up speed! Fuel: {self.fuel_level}%")

        else:

            print("Cannot accelerate: no fuel!")

    def brake(self):

        print(f"The {self.make} {self.model} slows down.")

    def refuel(self, amount):

        self.fuel_level = min(100, self.fuel_level + amount)

        print(f"Refueled to {self.fuel_level}%")

# Uso

mi_coche = Car("Toyota", "Corolla", 2020, "blue")

mi_coche.start_engine()
```

```
mi_coche.accelerate()
```

```
mi_coche.brake()
```

```
mi_coche.refuel(30)
```

Conclusión

- **Definición de clase** describe la estructura.
- **Atributos** guardan el estado de cada instancia.
- **Métodos** describen comportamientos y usan `self` para acceder a atributos.
- `__init__` inicializa objetos en el momento de la creación.

Funciones en la vida real

- **Problema:** Copiar y pegar el mismo código en varias partes vuelve el proyecto desordenado e ineficiente.
- **Solución:** Usar **funciones** para organizar y simplificar el código.
- **Analogía:** Las funciones son como **recetas de un libro de cocina**: defines los pasos una vez y luego puedes reutilizarlos siempre que los necesites.
- **Ejemplo:** Una función que calcule el área de un rectángulo recibe la **longitud** y el **ancho** como parámetros, y devuelve el área como resultado.
- **Ventajas de las funciones:**
 - Reutilización del código sin repetirlo.
 - Organización clara, como un libro de recetas frente a notas sueltas.
 - Posibilidad de usar nombres significativos y comentarios para mejorar la legibilidad.
 - Modularidad: cada función realiza una tarea específica.
 - Facilidad de prueba y reducción de errores.

En resumen: **Las funciones son bloques de construcción reutilizables que hacen el código más limpio, organizado, modular y fácil de mantener.**

Buenas prácticas al escribir funciones en Python

1. Convenciones de nomenclatura (nombres de funciones):

- Usar **snake_case** → letras minúsculas y palabras separadas por guiones bajos.
- Empezar con **verbos**: `calculate`, `get`, `filter`, `validate`, `process`.
- Ser **específico**: mejor `get_user_profile_data` que `get_data`.
- Mantener vocabulario consistente.
- Priorizar claridad sobre brevedad: `calculate_average` es más claro que `calc_avg`.
- Evitar nombres genéricos como `f` o `x`.

Funciones booleanas: usar prefijos como `is_`, `has_`, `can_`.

```
def is_valid_email(email):  
    # ... (Lógica de validación)  
    return True # O False, según la validación
```

```
def has_sufficient_balance(account):  
    # ... (Lógica de comprobación de saldo)  
    return True # O False, según el saldo
```

```
def can_access_resource(user, resource):  
    # ... (Lógica de permisos)  
    return True # O False
```

2. **Funciones internas:** usar guión bajo al inicio (`_nombre_funcion`) si no deben usarse fuera del módulo.

2. Docstrings (documentación de funciones):

- Son como el **manual de instrucciones** de la función.
- Deben incluir:
 - **Propósito** (qué hace y qué problema resuelve).
 - **Argumentos** (con nombres, tipos de datos (int, float...) y explicación).
 - **Valor de retorno** (tipo y significado).
 - **Excepciones** (si puede lanzar errores).
 - **Ejemplos** de uso (opcional).
- Incluir **type hints** mejora la claridad y la compatibilidad con editores.

Ejemplo de docstring completo:

```
def calculate_monthly_payment(principal: float, interest_rate: float, loan_term_years: int) -> float:
```

```
    """
```

```
    Calculates the monthly payment for a fixed-rate loan.
```

```
    Args:
```

```
        principal (float): The total amount borrowed.
```

```
        interest_rate (float): The annual interest rate (as a decimal).
```

```
        loan_term_years (int): The loan term in years.
```

```
    Returns:
```


float: The fixed monthly payment.

Raises:

ValueError: If interest_rate is negative or loan_term_years <= 0.

Example:

```
>>> calculate_monthly_payment(100000, 0.05, 30)

536.82

"""

# Ejemplo de implementación básica

monthly_rate = interest_rate / 12

months = loan_term_years * 12

payment = principal * (monthly_rate * (1 + monthly_rate) **
months) / ((1 + monthly_rate) ** months - 1)

return payment
```

En resumen:

- Usa nombres claros, consistentes y específicos.
- Prefiere funciones cortas, enfocadas en una sola tarea.
- Documenta con **docstrings detallados** que incluyan propósito, entradas, salidas y excepciones.
- Mantén los docstrings actualizados para facilitar el mantenimiento y la colaboración.

1. Longitud de funciones: equilibrio

- Una función debe ser **concisa pero completa**.
- Ideal: entre **10 y 20 líneas** (cabén en una pantalla sin hacer scroll).
- Funciones demasiado largas → difíciles de leer, probar y mantener.
- Funciones demasiado cortas → poco claras, falta de contexto.

2. Principio de Responsabilidad Única (SRP)

- Cada función debe hacer **una sola cosa** y hacerla bien.
- Señales para dividir una función:
 - **Múltiples responsabilidades**: obtener datos, procesar y guardar → separar.
 - **Sobrecarga cognitiva**: si cuesta entender la lógica → dividir.
 - **Dificultad para probar**: funciones largas = difíciles de testear.

3. Evitar efectos secundarios

- Problema común: **modificar variables globales** dentro de funciones → código impredecible.
- Mejor: pasar datos como **argumentos** → funciones más puras, autónomas y predecibles.
- Evitar que funciones impriman, guarden datos o lancen errores directamente → separar responsabilidades en funciones específicas.

4. Ejemplo de mala práctica (efecto secundario)

```
global_sum = 0 # Variable global

def calculate_mean_with_side_effect(numbers):

    global global_sum # Modifica la variable global

    global_sum = sum(numbers)

    return global_sum / len(numbers)

calculate_mean_with_side_effect([5, 3, 4, 1, 1])

# Resultado: 2.8
```

✗ Esta función no solo calcula la media, también **cambia el estado global**, volviéndola menos predecible.

5. Ejemplo de buena práctica (función pura)

```
def calculate_mean(numbers):

    """Calculates the mean of a list of numbers."""

    return sum(numbers) / len(numbers)

calculate_mean([5, 3, 4, 1, 1])

# Resultado: 2.8
```

✓ Función pura: su salida depende solo de la entrada. Es fácil de **razonar, probar y reutilizar**.

6. Beneficios de las funciones puras

- **Predecibles y fáciles de probar** → solo dependen de la entrada.
- **Reutilizables** en distintas partes del programa sin causar errores.
- Aptas para **procesamiento paralelo** (no modifican datos compartidos).
- Favorecen un código **más limpio, mantenible y colaborativo**.

En conclusión:

Mantén tus funciones **cortas, enfocadas en una sola tarea y sin efectos secundarios**.

Prioriza el **paso de argumentos** y la creación de **funciones puras** para lograr código más robusto y elegante.

EJEMPLO MÍO

Ejemplo mío

```
def calculate_diameter_circle(radius: float) -> float:
```

```
    """
```

```
    Debe calcular el diámetro de un círculo (diámetro = radius * 2)
    y devolverlo.
```

```
    Así como no hay ningún requisito para que el radio sea un número
    entero, usted debe asumir un retorno de float.
```

```
    Se le han dado instrucciones de que si el usuario envía un radio
    negativo, la función debe devolver -1 para indicar un error, ya que
    un radio negativo no es una situación común.
```

```
    """
```

```
    diameter = radius*2
```

```
    if radius >= 0:

        return diameter

    else:

        return -1 #es como valor de error

### Ejemplo


diameter_1 = calculate_diameter_circle(7)

print(diameter_1)


diameter_2 = calculate_diameter_circle(-3)

print(diameter_2)


diameter_3 = calculate_diameter_circle(2.5)

print(diameter_3)


diameter_4 = calculate_diameter_circle(1000000)

print(diameter_4)


diameter_4 = calculate_diameter_circle(0)

print(diameter_4)


salida:

14

-1
```

5.0

2000000

0

ACTIVIDADES Escenario: Alcance de funciones

Code Wizards, Inc, una empresa líder en análisis de datos, ha solicitado su ayuda para demostrar los conceptos de funciones y alcance a los estudiantes que asistirán a un evento del día de la carrera en su ubicación. El equipo de formación le ha proporcionado los siguientes ejemplos. Trabjará con ellos para realizar la prueba. Dado que tratará con principiantes, intentará seguir los ejemplos tal y como están escritos y evitará material avanzado (como sugerencias de tipo) o mejorar el código cuando practique la demostración.

En Scoping out Functions, usted deberá:

- Crear una función sin parámetros y sin valor de retorno.
- Crear una función con parámetros y sin valor de retorno.
- Crear una función sin parámetros y con valor de retorno.
- Crear una función con parámetros y un valor de retorno.
- Crear un ejemplo para mostrar el ámbito de las variables.
- Crear un ejemplo para demostrar la importación de módulos.

1. Ejercicio 1: Sin parámetros, sin valor de retorno

Instrucciones:

En este ejemplo, usted escribirá una función que no toma parámetros y mostrará un mensaje estándar.

1. El código inicial tiene una llamada a una función, `happy_birthday`, que usted escribirá.
2. Escriba la definición de la función, `happy_birthday`, que no toma parámetros (1 línea de código). Evite las sugerencias de tipo para los parámetros.
3. La función debe imprimir `Happy Birthday!` para el usuario (1 línea de código). La función no debe devolver ningún valor. Asegúrese de que las mayúsculas coinciden exactamente (H mayúscula y B mayúscula) y de que incluye el signo de exclamación.
4. Ejecute el programa. El programa mostrará siempre el mismo mensaje.

```
1 def happy_birthday():  
2     "happy_birthday no tiene parámetros"  
3     return "Happy Birthday!"  
4  
5 # Code to call the function  
6 happy_birthday()
```

print mejor

Ejecutar

Restablecer

✗ Incorrecto

Comentarios: La función 'happy_birthday' debe incluir una sentencia print que utilice las palabras '¡Feliz cumpleaños!'.

2. Ejercicio 2: Parámetros, pero sin valor de retorno

Instrucciones

En este ejemplo, usted escribirá una función que toma dos parámetros y mostrará un mensaje personalizado.

1. El código inicial tiene una llamada a una función, `happy_birthday`, que usted escribirá.
2. Escriba la definición de la función, `happy_birthday`, que toma dos parámetros, el primero llamado `age` y el segundo llamado `name` (1 línea de código). Evite las sugerencias de tipo para los parámetros.
3. La función debe imprimir `Happy Birthday <name> and congratulations on turning <age> years old!` para el usuario (1 línea de código). Por ejemplo, si pasa los valores 22 para la edad y Nora para el nombre, el programa debe mostrar `Happy Birthday Nora and congratulations on turning 22 years old!`. La función no debe devolver ningún valor. Asegúrese de que el espaciado es coherente con los requisitos (por ejemplo, "Feliz cumpleaños Nora" tiene dos espacios entre "Cumpleaños" y "Nora" cuando debería tener uno).
4. Ejecute el programa. ASÍ COMO ESTÁ ESCRITO, siempre mostrará el mismo mensaje, pero podría modificarse para aceptar entradas en lugar de utilizar siempre los mismos valores.

```
1 def happy_birthday(age, name):
2
3     print("Happy Birthday", name, "and congratulations on turning", age, "years old")
4
5
6 # Code to call the function
7 happy_birthday(22, "Nora")
```

[Ejecutar](#)[Restablecer](#)

✓ Correcto

Comentarios: ¡Buen trabajo! Tu función muestra correctamente el mensaje de cumpleaños utilizando los parámetros de la función 'happy_birthday'.

Tu salida: ¡Feliz cumpleaños Nora y felicidades por cumplir 22 años!

3. Ejercicio 3: Sin parámetros, pero con valor de retorno

Instrucciones

En este ejemplo, crearás una función para generar un número de la suerte. Al declararla como una función, este código puede ser reutilizado en otras partes del programa. Al tener la definición en un solo lugar, le permitiría cambiar la funcionalidad en muchos lugares cambiando una línea en la función.

1. Comience escribiendo la definición de función para una función `get_lucky_number`. Recuerde, esta función no debe requerir ningún parámetro de entrada (una línea de código). Evite las sugerencias de tipo para el valor de retorno.
2. A continuación, la función debe devolver el valor de `lucky_num` (una línea de código). El valor ya ha sido cargado; su tarea es devolver el valor al programa principal.
3. Ejecute el programa. Se generará un número aleatorio en el rango (1 a 100, como se define en la función). Esto permite escribir la lógica una vez y reutilizarla muchas veces en el programa.

```
1 import random
2
3 def get_lucky_number():
4     lucky_num = random.randint(1,100)
5     return lucky_num
6
7 # Get a lucky number between 1 and 100
8 lucky_number = get_lucky_number()
9
10 print("Your lucky number is:", lucky_number)
```

[Ejecutar](#)[Restablecer](#)

✓ **Correcto**

Comentarios: ¡Buen trabajo! Tu función genera e imprime correctamente un número de la suerte.

Tu salida:

Tu número de la suerte es: 18

4. Ejercicio 4: Parámetros y valor de retorno

Instrucciones

En este ejemplo, usted escribirá una función para calcular el total descontado basado en si un usuario es miembro del club de descuentos o no.

1. Empiece escribiendo la definición de la función `calc_sale_price`. Esta función aceptará dos parámetros de entrada (una línea de código). El primero, `amount`, es un número que representa el importe de la compra. El segundo, `member`, es una variable booleana que representa si el usuario es socio (True) o no (False). Evite las sugerencias de tipo para los parámetros y el tipo de retorno.
2. Redondee `amount` a dos decimales utilizando la función integrada `round`. Guarde el resultado en la variable `amount`.
3. La función debe devolver el valor de `amount`.
4. Ejecute el código. Se mostrarán los importes descontados.

```
1  def calc_sale_price(amount, member):
2      if member:
3          # Members receive a 15% discount (0.15)
4          amount = amount - (amount * 0.15)
5      else:
6          # Non-members get a 5% discount (0.05)
7          amount = amount - (amount * 0.05)
8
9          # Round amount to two decimal places
10         # Save back in the amount variable
11         amount = round(amount, 2)
12
13         # Return amount to the main program
14         return amount
15
16     # Example price (already provided)
17     full_price = 150.50
18
19     # Call function for members
20     member_price = calc_sale_price(full_price, True)
21     print("Member price:", member_price)
22
23     # Call function for non-members
24     non_member_price = calc_sale_price(full_price, False)
25     print("Non-member price:", non_member_price)
```

Ejecutar

Restablecer

✓ Correcto

Comentarios: ¡Buen trabajo! Su función muestra correctamente el precio tanto para los miembros como para los no miembros.

Su resultado: Precio para socios: 127,92

Precio para no socios: 142,97

5. Ejercicio 5: Ámbito de aplicación

Instrucciones

En este ejemplo, explorará el concepto de alcance.

1. Ejecute el código siguiente y observe el error. La primera llamada a una función, `display_color_works()`, se ejecutará correctamente. La segunda, `display_color_failure()`, resultará en un `NameError`.
2. Hay varias maneras de corregir esto.
 - a. Una forma es comentar la línea que llama a `display_color_failure`. Esto no soluciona el problema, pero evita el error.
 - b. La forma preferida es establecer `shirt_color` fuera de la función `display_color_works`, en el programa principal, y pasar el valor a ambas funciones.
 - c. Una tercera forma es atrapar el error utilizando un bloque try/catch.
 - d. Una cuarta forma es declarar la variable como una variable global, pero como se discutió en una lectura anterior, esto se considera una mala práctica.
3. Comente la llamada a la función `display_color_failure` y vuelva a ejecutar el programa. Comprueba que el error desaparece.

```
1  shirt_color = 'Pink' # se define fuera para no causar error
2
3  def display_color_works():
4      print("First shirt color is:", shirt_color)
5
6  def display_color_failure():
7      # Try to access 'color' directly (this will cause an error)
8      print("Your shirt color is:", shirt_color)
9
10 # The shirt_color variable is in scope in this function
11 display_color_works()
12
13 # The shirt_color variable is not in scope in this function
14 display_color_failure()
```

[Ejecutar](#)[Restablecer](#)

✓ Correcto

Comentarios: ¡Buen trabajo! Has resuelto el `NameError` y tu código contiene la salida esperada.

La salida esperada debería contener: El color de la primera camisa es: Rosa

Tu salida: El color de la primera camisa es: Rosa

El color de tu camisa es Rosa

6. Ejercicio 6: Importar un módulo

Instrucciones

En este ejemplo, examinarás el uso de funciones de otro módulo.

1. Usted ha creado un archivo llamado `menus.py` almacenado en la misma carpeta que su programa actual. Este módulo contiene una función, `display_menu`, que ha sido escrita. No toma argumentos y devuelve un valor numérico.
2. Importa el módulo `menus` para que su contenido esté disponible en tu programa.
3. Introduzca el código para llamar al módulo `display_menu` en el módulo de menús. El resultado se guardará en la variable `user_choice`.
4. Este código requiere un archivo `menus.py`. Funcionará en la plataforma Coursera, pero no funcionaría en un editor de código separado en su sistema. Diseñar de esta manera permite a otro programador modificar el módulo de menús independientemente de su código, y su programa se beneficiará de la última actualización.

```
1 import menus
2
3 # Call the display_menu function in the menus module
4 user_choice = menus.display_menu()
5
6 print(user_choice)
```

[Ejecutar](#)[Restablecer](#)

Correcto

Comentarios: ¡Buen trabajo! Has implementado con éxito el comando de importación.

Tu salida: Bien hecho, tu código funciona correctamente.

CUESTIONARIO

1. Imagina que estás creando un programa para gestionar la colección de libros de una biblioteca. Decides crear una función llamada `add_book`. ¿Qué información sería esencial incluir como argumentos para que esta función funcione eficazmente? Seleccione todo lo que corresponda.

☒ El título del libro

☒ **Correcto**

Correcto El título es un dato fundamental para identificar un libro.

☐ La fecha y hora actuales

☒ El autor del libro

☒ **Correcto**

Correcto El autor es crucial para catalogar y buscar el libro.

☒ Año de publicación del libro

☒ **Correcto**

Correcto El año de publicación ayuda a distinguir las distintas ediciones y proporciona contexto.

2. Estás desarrollando un juego en el que los jugadores ganan puntos por completar diferentes tareas. Tienes una función llamada `calculate_bonus_points` que toma la puntuación actual del jugador como argumento y calcula puntos de bonificación adicionales basados en ciertas condiciones. ¿Dónde debe almacenar la puntuación total acumulada del jugador para garantizar que se puede acceder y actualizar tanto dentro como fuera de la función `calculate_bonus_points` y otras funciones del juego? Seleccione la mejor respuesta.

☐ Como valor de retorno de la función `calculate_bonus_points`.

☐ Como argumento de la función `calculate_bonus_points`.

☒ Como variable global.

☐ Como variable local dentro de la función `calculate_bonus_points`.

☒ **Correcto**

Correcto El almacenamiento de la puntuación total del jugador como una variable global permite que sea accesible y modificada por cualquier función en su juego, incluyendo `calculate_bonus_points`. Esto asegura que la puntuación se mantiene constante a lo largo de todo el juego.

3. Estás trabajando en un programa Python que calcula el coste total de los artículos de un carrito de la compra. Tienes una función llamada `calculate_total_cost` que toma los precios de los artículos como una lista y aplica un descuento si es aplicable. También tienes una variable global llamada `SALES_TAX_RATE` que almacena la tasa actual del impuesto sobre las ventas. Dado que la variable `SALES_TAX_RATE` ya está declarada globalmente, ¿cómo debería accederse a ella en la función `calculate_total_cost`? Seleccione la mejor respuesta.

- ☐ Declare `SALES_TAX_RATE` como argumento de la función `calculate_total_cost` y acceda a él como parámetro.
- ☒ Acceda a `SALES_TAX_RATE` directamente desde `calculate_total_cost`.
- ☐ Declare `SALES_TAX_RATE` dentro de la función `calculate_total_cost` y acceda a ella utilizando la palabra clave global.
- ☐ Declare `SALES_TAX_RATE` dentro de la función `calculate_total_cost` y acceda a ella directamente.

✓ **Correcto**

Correcto Declarar `SALES_TAX_RATE` fuera de cualquier función la convierte en una variable global, accesible en todo el programa, incluso dentro de la función `calculate_total_cost`.

4. Estás diseñando un juego de acuario virtual y decides utilizar clases para representar las distintas especies de peces. Para gestionar estas diversas especies de manera eficiente y permitir acciones comunes como alimentar a todos los peces del acuario con un solo comando, sin dejar de tener en cuenta sus comportamientos de alimentación únicos, ¿qué par de conceptos de programación orientada a objetos sería MÁS ventajoso implementar? Seleccione la mejor respuesta.

- ☐ Datos y métodos: Definición de las propiedades (por ejemplo, nombre, tamaño, dieta) y acciones (por ejemplo, nadar, comer) de cada especie de pez dentro de sus respectivas clases.
- ☐ Abstracción y reutilización: Simplificar la representación de los peces centrándose en las características esenciales y creando múltiples instancias de diferentes especies de peces a partir de sus respectivas clases.
- ☒ Herencia y polimorfismo: Crear una clase base `Fish` con atributos y comportamientos comunes y, a continuación, crear clases especializadas para cada especie (por ejemplo, `Clownfish`, `Angelfish`) que hereden de `Fish` e implementen su lógica de alimentación específica. Esto permite tratar a todos los peces de manera uniforme para las acciones generales, respetando al mismo tiempo sus necesidades individuales.
- ☐ Encapsulación y aleatorización: Ocultación de los datos internos de los objetos peces y generación de atributos aleatorios para cada instancia de pez.

✓ **Correcto**

Correcto La herencia le permite crear tipos de peces especializados con características compartidas, mientras que el polimorfismo le permite interactuar con ellos a través de una interfaz común (por ejemplo, alimentar a todos los peces independientemente de su tipo específico), acomodando sus comportamientos únicos.

5. Está analizando un gran conjunto de datos y se da cuenta de que hay bloques de código idénticos que se repiten varias veces a lo largo del script. Estos bloques de código tienen diferentes propósitos. ¿Cuál es el MEJOR enfoque que aprovecha las funciones para agilizar el código y evitar la redundancia? Seleccione la mejor respuesta.
- ☐ Ignore los bloques de código repetidos y continúe, abordándolos más adelante si es necesario.
 - ☒ Encapsule cada bloque de código repetido en su propia función independiente y llame a esas funciones según sea necesario.
 - ☐ Combina todos los bloques de código repetidos en una única función larga, evitando la creación de cualquier otra función.
 - ☐ Copiar y pegar los mismos segmentos de código para cada tarea de análisis de datos, garantizando la coherencia.

☒ **Correcto**

Correcto Esto promueve la modularidad, la reutilización y la mantenibilidad.

6. Estás trabajando en un proyecto de equipo en el que tienes que escribir una función para validar las contraseñas de los usuarios. Las contraseñas deben cumplir ciertos criterios, como tener una longitud mínima, contener letras mayúsculas y minúsculas, e incluir al menos un dígito. Debe devolver Verdadero o Falso. ¿Cuál de las siguientes definiciones de función cumple MEJOR con las convenciones de nomenclatura recomendadas y las mejores prácticas para escribir funciones Python? Seleccione la mejor respuesta.

- ☐ `def ValidatePassword(password)`
- ☒ `def is_valid_password(password)`
- ☐ `def is_valid_password(pwd)`
- ☐ `def check_password_validity(password)`

☒ **Correcto**

Correcto Este nombre de función es descriptivo, sigue snake_case, utiliza un prefijo booleano para indicar que devuelve un valor Verdadero/Falso y utiliza un nombre de parámetro completo y claro.

Creación de clases personalizadas

Clases personalizadas en Python

1. **Problema inicial:**
En proyectos grandes, el código puede volverse desordenado y difícil de manejar.
2. **Solución: Clases personalizadas**

- Funcionan como **planos** o **bloques de construcción** para crear objetos.
- Permiten **encapsular datos (atributos)** y **funciones (métodos)** relacionados en una sola unidad reutilizable.
- Mejoran la **organización, modularidad y reutilización** del código.

3. Atributos y métodos

- **Atributos** = propiedades o características de un objeto (ej.: color, tamaño, nombre).
- **Métodos** = comportamientos o acciones del objeto (ej.: mover, atacar, calcular).

4. Ventajas clave

- Código más **legible y mantenible**.
- Cambios localizados (modificar un método dentro de la clase afecta a todas las instancias).
- Reutilización: una clase definida puede crear múltiples instancias independientes.

5. Cómo empezar

- Identifica los **conceptos principales (sustantivos)** de tu proyecto → por ejemplo, **Jugador**, **Enemigo**, **Arma**.
- Define los **atributos** (propiedades) y **métodos** (acciones) relevantes para cada concepto.

En resumen: **Las clases personalizadas ayudan a dividir proyectos grandes en partes más pequeñas, claras y reutilizables, representando entidades del programa con atributos y comportamientos propios.**

Resolución de problemas con funciones en Python

1. Funciones como herramienta de resolución de problemas

- Dividen un problema complejo en partes **más pequeñas y manejables** (descomposición).
- Encapsulan tareas en **bloques modulares reutilizables**.
- Ocultan detalles internos: funcionan como **cajas negras** que solo dan resultados.

2. Beneficios principales

- **Reutilización:** evitar repetir lógica en distintas partes del programa.
- **Consistencia y menos errores:** una sola definición centralizada.
- **Pruebas más fáciles:** cada función puede evaluarse de forma aislada.
- **Colaboración en equipo:** varios programadores pueden trabajar sin pisarse.

3. El arte de la descomposición (Principio SRP)

- El **Principio de Responsabilidad Única (SRP)** dice que una función debe hacer **una sola cosa bien**.
- Ejemplo: en una red social tipo Twitter, en lugar de una única función enorme `post_tweet()`, conviene dividirla en **subfunciones especializadas** (validar longitud, validar archivos, etc.).

Ejemplo en código: Validaciones de un tweet

```
def is_valid_tweet_length(tweet_text):  
    """Valida la longitud del tweet."""  
  
    if not tweet_text:  
        return False # El tweet no puede estar vacío
```

```

    if len(tweet_text) > 280:

        return False # El tweet es demasiado largo (más de 280
caracteres)

    return True

def are_valid_media_files(media_files):

    """Valida los archivos multimedia (comprobación básica)."""

    if media_files is None:

        return True # No pasa nada si no hay archivos multimedia

    elif isinstance(media_files, list):

        return True # Una lista de archivos es válida en esta
comprobación básica

    else:

        return False # Cualquier otro formato no es válido

```

4. Ventajas del enfoque modular

- **Mantenimiento sencillo:** cambiar solo la función específica que necesite ajustes.
- **Capacidad de prueba:** permite crear **tests unitarios** por separado para cada función.
- **Reutilización:** las funciones como `is_valid_tweet_length()` pueden usarse en diferentes contextos del programa.

Conclusión: Aplicar funciones para dividir problemas grandes en **unidades pequeñas, claras y reutilizables** da como resultado un código más **modular, mantenible y fácil de probar**.

Aprovechar el potencial de la descomposición funcional

1. Funciones bien estructuradas

- Hacen que el código sea **más legible, modular y mantenible**.
- Los errores se pueden localizar con mayor precisión, ya que cada función tiene una **responsabilidad clara**.
- Facilitan la **reutilización en múltiples contextos**: interfaz gráfica, API, automatización de tareas, etc.

2. Ejemplo práctico 1: Publicación de tweets

- En lugar de un `post_tweet()` enorme y desordenado, se divide en funciones más pequeñas:
 - `is_valid_tweet_length()` → valida longitud.
 - `are_valid_media_files()` → valida archivos multimedia.
- Esto permite cambiar solo lo necesario sin afectar al resto del sistema.

3. Ejemplo práctico 2: Análisis de reseñas de restaurantes

- Se construye una **canalización de análisis de sentimientos** usando funciones modulares:

```
from textblob import TextBlob

from nltk.corpus import stopwords

from collections import Counter

def preprocess_review(review):
```

```

    """Removes stopwords, punctuation, and converts to lowercase."""
    # ... (Implementation details)

    return preprocessed_review

def analyze_sentiment(preprocessed_review):
    """Calculates polarity and subjectivity using TextBlob."""
    # ... (Implementation details)

    return polarity, subjectivity

def extract_keywords(preprocessed_review):
    """Identifies the most frequent nouns and adjectives."""
    # ... (Implementation details using NLTK)

    return keywords

def categorize_review(polarity, subjectivity, keywords):
    """Classifies reviews based on polarity, subjectivity, and
    keywords."""
    # ... (Implementation details with custom rules)

    return category

```

4.

- **Flujo de trabajo:**

- `preprocess_review` → limpia y normaliza el texto.
- `analyze_sentiment` → mide polaridad (-1 a 1) y subjetividad (0 a 1).

- `extract_keywords` → identifica aspectos clave (ej. "servicio", "rápido").
- `categorize_review` → clasifica como **positiva, negativa o neutra** según reglas.

5. Beneficios de este enfoque modular

- **Mantenimiento:** puedes mejorar o arreglar una función sin romper todo el sistema.
- **Pruebas unitarias:** cada función se prueba por separado.
- **Reutilización:** las funciones pueden servir en otros proyectos o contextos.
- **Colaboración:** equipos diferentes trabajan en módulos distintos (ej. validaciones, UI, análisis de datos).

Conclusión: La descomposición funcional convierte problemas complejos en bloques manejables, reutilizables y fáciles de probar. Es la clave para mantener proyectos Python grandes **organizados, escalables y colaborativos**.

ACTIVIDAD Bloque de código funcional: Herramienta de conversión de temperatura (3 funciones) ### MUY BUEN EJERCICIO

```
def celsius_to_fahrenheit(celsius: float) -> float:

    """ Toma una temperatura en grados Celsius y la convierte a
    grados Fahrenheit. """

    fahrenheit = (celsius * 9/5) + 32

    return fahrenheit


def fahrenheit_to_celsius(fahrenheit: float) -> float:
```

```

    """ Toma una temperatura en Fahrenheit y la convierte a
    Celsius."""

    celsius = (fahrenheit - 32) * 5/9

    return celsius

def convert_temperature(temperature, unit): # función 3

    """Toma un valor de temperatura y una unidad ('C' para Celsius o
    'F' para Fahrenheit). Llama a la función de conversión apropiada
    basada en la unidad y devuelve la temperatura convertida."""

    if unit == "C":

        return celsius_to_fahrenheit(temperature) ## llama a la
función 1 con el argumento de temperatura

    elif unit == "F":

        return fahrenheit_to_celsius(temperature) ## ## llama a la
función 2 con el argumento de temperatura

    else:

        print("Introduce C o F")

# EJEMPLO DE USO

temperature_c = 25

temperature_f = 77

converted_f = convert_temperature(temperature_c, 'C')

converted_c = convert_temperature(temperature_f, 'F')

```

```
print(f"{temperature_c}°C is equal to {converted_f}°F")  
print(f"{temperature_f}°F is equal to {converted_c}°C")
```

SALIDA:

25°C is equal to 77.0°F

77°F is equal to 25.0°C

Dividir y conquistar con la modularidad en Python

En Python, las **funciones** son como recetas: pasos claros y reutilizables para realizar tareas específicas.

La **modularidad** significa dividir el código en pequeñas unidades (funciones) que se combinan para crear programas organizados y fáciles de mantener.

Ventajas de la modularidad:

- Mejor **legibilidad**: cada función tiene un propósito claro.
- **Reutilización** de código sin repetir instrucciones.
- **Depuración más sencilla**: puedes localizar errores en funciones específicas.
- En equipos, permite trabajar en paralelo con menos conflictos.

Valores por defecto en funciones

Los **parámetros por defecto** permiten manejar casos comunes sin necesidad de escribir varias funciones.

Ejemplo: cálculo de área de un rectángulo (o cuadrado si los lados son iguales).

```
def calculate_area(width=1, height=1):  
    """Calcula el área de un rectángulo, con valores por defecto de  
    1x1."""
```

```
def calculate_area(width, height):  
    area = width * height  
    return area  
  
# Uso sin argumentos → cuadrado 1x1  
  
result = calculate_area()  
print(result) # Output: 1  
  
# Uso con valores → rectángulo 5x3  
  
result = calculate_area(5, 3)  
print(result) # Output: 15
```

Precaución con argumentos mutables

- Python evalúa los argumentos por defecto **solo una vez** al definir la función.
- Si usas listas o diccionarios como valores por defecto, pueden compartirse entre llamadas → comportamiento inesperado.
- Solución: usar **None** y crear el objeto dentro de la función.

Parámetros opcionales

Son aquellos que tienen un valor por defecto, lo que los hace flexibles.

Ejemplo: función para saludar:

```
def greet(name, greeting="Hello"):  
    """Saluda a una persona con un mensaje opcional."""
```



```
print(greeting, name)

# Caso usando el valor por defecto
greet("Alice")

# Output: Hello Alice

# Personalizando el saludo
greet("Bob", "Good morning")

# Output: Good morning Bob

greet("Carol", "Howdy")

# Output: Howdy Carol
```

Conclusión

Los **valores por defecto** y los **parámetros opcionales** hacen que las funciones sean:

- **Más flexibles** para manejar distintos escenarios.
- **Más simples de usar**, al evitar repetir código.
- **Más claras y modulares**, lo que mejora la colaboración y el mantenimiento del programa.

Más allá de lo básico: Técnicas avanzadas en funciones

Las funciones en Python no solo aceptan **parámetros obligatorios y opcionales**, también pueden manejar:

1. Argumentos con palabras clave (keyword arguments)

Permiten especificar explícitamente qué valor corresponde a qué parámetro, mejorando la legibilidad:

```
def greet(name, greeting="Hello"):

    print(greeting, name)

greet(name="David", greeting="Salutations")

# Output: Salutations David
```

2. Número variable de argumentos (*args y **kwargs)

- ***args**: agrupa argumentos posicionales en una **tupla**.
- ****kwargs**: agrupa argumentos con nombre en un **diccionario**.

```
def flexible_function(*args, **kwargs):

    print("Positional arguments:", args)

    print("Keyword arguments:", kwargs)

flexible_function(1, 2, 3, name="Alice", age=30)

# Positional arguments: (1, 2, 3)

# Keyword arguments: {'name': 'Alice', 'age': 30}
```

3. Ejemplo completo: creación de perfiles de usuario

```
def create_user_profile(name, age, occupation="Student",
interests=None):
```

```
"""
Creates a user profile with optional interests.

Args:
    name (str): The user's name (required).
    age (int): The user's age (required).
    occupation (str, optional): The user's occupation (defaults
to "Student").
    interests (list, optional): A list of the user's interests
(defaults to None).
"""

if interests is None: # Initialize if None to avoid mutable
default issues

    interests = []

profile = {
    "name": name,
    "age": age,
    "occupation": occupation,
    "interests": interests
}

return profile

# Ejemplos de uso
```

```
user1 = create_user_profile("Alice", 25, "Software Engineer",
["Coding", "Hiking"])

user2 = create_user_profile("Bob", 18) # Ocupación por defecto y
sin intereses

user3 = create_user_profile("Carol", 30, interests=["Gardening",
"Reading"])

print(user1)

print(user2)

print(user3)
```

Salida:

```
{'name': 'Alice', 'age': 25, 'occupation': 'Software Engineer',
'interests': ['Coding', 'Hiking']}

{'name': 'Bob', 'age': 18, 'occupation': 'Student', 'interests': []}

{'name': 'Carol', 'age': 30, 'occupation': 'Student', 'interests':
['Gardening', 'Reading']}
```

Puntos clave del ejemplo:

- **Parámetros obligatorios:** `name`, `age`.
- **Parámetros opcionales con valores por defecto:** `occupation`, `interests`.
- **Uso de `None` como valor por defecto:** evita problemas con objetos mutables como listas.
- **Argumentos de palabra clave (`interests=...`):** mejoran la claridad al llamar la función.
- **Docstring claro:** documenta propósito, argumentos y valores de retorno.

Con estas técnicas avanzadas, tus funciones serán:

- **Más flexibles** (manejan múltiples escenarios).
- **Más seguras** (evitan errores con valores mutables).
- **Más legibles** (gracias a docstrings y keyword arguments).
- **Más reutilizables y robustas**, ideales para proyectos grandes y colaborativos.

ACTIVIDAD: Reto de código funcional: Hacer un sándwich

```
def make_sandwich(bread_type: str, filling: str, cheese: str =  
"ninguno", toasted: bool = False) -> str:
```

```
    """
```

Construye una descripción de un sándwich con pan, relleno, queso
opcional y tostado opcional.

Args:

bread_type (str): Tipo de pan.

filling (str): Relleno principal.

cheese (str, optional): Tipo de queso. Por defecto
"ninguno".

toasted (bool, optional): Si el sándwich está tostado. Por
defecto False.

Returns:

str: Descripción del sándwich.

```
    """
```

```
    description = f"Making a" # hace uso de los f strings para
ponerlo todo bonito
```

```
    if toasted is True:
```

```
        description += " toasted"
```

```
    description += f" {bread_type} sandwich with {filling}"
```

```
    if cheese.lower() != "ninguno":
```

```
        description += f" and {cheese} cheese"
```

```
    description += "."
```

```
    return description
```

```
# Ejemplo de uso
```

```
print(make_sandwich("wheat", "turkey", "cheddar", True))
```

```
print(make_sandwich("rye", "ham"))
```

SALIDA:

Making a toasted wheat sandwich with turkey and cheddar cheese.

Making a rye sandwich with ham.

ACTIVIDAD Y CUESTIONARIO DE LA CLASE TEMPERATURE DATA (MIRAR CÓDIGO EN PYTHON)

1. ¿Cuál es el propósito principal del método `__init__` en `TemperatureData`? Selecciona la mejor respuesta.

- ☐ Para realizar cálculos complejos relacionados con los datos de temperatura
- ☒ Para inicializar los atributos del objeto (como valores de temperatura, unidades, etc.) cuando se crea una nueva instancia.
- ☐ Crear una representación visual de los datos de temperatura
- ☐ Para imprimir un mensaje de bienvenida al usuario

✓ **Correcto**

Correcto En la clase `TemperatureData`, se inicializan el nombre y las lecturas.

2. En la clase `TemperatureData`, ¿qué representa el parámetro `self` dentro de un método de clase? Selecciona la mejor respuesta.

- ☐ Una forma de acceder a las variables globales.
- ☐ Un marcador de posición para la entrada del usuario.
- ☒ Una referencia a la instancia actual de la clase.
- ☐ Una referencia a la propia clase.

✓ **Correcto**

Correcto El método puede acceder y modificar los atributos y el comportamiento del objeto específico al que se llama haciendo referencia a `self`.

3. Dado el código para crear un objeto `TemperatureData` en el Notebook de Jupyter, ¿cómo crearías un segundo sensor llamado "West Forest Road Sensor" con lecturas de temperatura de 76, 71, 64, 63, y 86? Selecciona la mejor respuesta.

- ☐ `second_sensor = TemperatureData("West Forest Road Sensor", 76, 71, 64, 63, 86)`
- ☒ `second_sensor = TemperatureData("West Forest Road Sensor", [76, 71, 64, 63, 86])`
- ☐ `second_sensor = "West Forest Road Sensor", [76, 71, 64, 63, 86]`
- ☐ `TemperatureData.new_sensor("West Forest Road Sensor", [76, 71, 64, 63, 86])`

✓ **Correcto**

Correcto Esto crea correctamente una nueva instancia de la clase `TemperatureData` con el nombre especificado y las lecturas de temperatura.

4. Si se le pidiera que modificara el método `calculate_average_temp` para que devolviera la temperatura media redondeada a un decimal, ¿cuál es la mejor manera de hacerlo? Seleccione la mejor respuesta.

- ☐ Crear un nuevo método para la salida formateada:

```
def calculate_rounded_average_temp(self):  
  
    return round(sum(self.readings) / len(self.readings), 1)
```

- ☒ Modifica el método `calculate_average_temp` para incluir el redondeo:

```
def calculate_average_temp(self):  
  
    return round(sum(self.readings) / len(self.readings), 1)
```

- ☐ Redondea el resultado directamente en la sentencia print:

```
average_temp = sensor.calculate_average_temp()  
  
print(f"Average temperature for sensor {sensor_name}: {round(average_temp,  
1)} degrees Fahrenheit")
```

- ☐ Redondea el resultado en el programa cuando se realiza el cálculo:

```
average_temp = round(sensor.calculate_average_temp(), 1)
```

✓ **Correcto**

Correcto Esto encapsula la lógica de redondeo dentro del propio método, asegurando un comportamiento consistente.

5. ¿Cuál de los siguientes métodos sería útil añadir a la clase `TemperatureData` para su posterior análisis? Seleccione todos los que correspondan.

☒ `convert_to_fahrenheit()` - Convierte las lecturas de temperatura de Celsius a Fahrenheit.

☒ **Esto no debería estar seleccionado**

No del todo. Este método sólo es relevante si las lecturas originales estaban en Celsius, lo que no se especifica en la definición de la clase actual.

☒ `calculate_median_temp()` - Calcula la temperatura media a partir de las lecturas.

☒ **Correcto**

Correcto La temperatura mediana puede proporcionar información valiosa sobre la tendencia central de los datos.

☐ `display_sensor_name()` - Muestra el nombre del sensor.

☒ `count_readings_above_threshold(threshold)` - Cuenta el número de lecturas por encima de un umbral especificado.

☒ **Correcto**

Correcto Este método permite un análisis flexible basado en umbrales de temperatura específicos.

CUESTIONARIO

1. ¿Cuál es la principal ventaja de utilizar clases personalizadas en Python? Elige la mejor respuesta.

- ☐ Hacen que tu código funcione más rápido.
- ☒ Le ayudan a organizar su código en unidades reutilizables y manejables.
- ☐ Permiten utilizar funciones preconstruidas de bibliotecas.
- ☐ Corrigen automáticamente los errores de su código.

✔ **Correcto**

Correcto Las clases personalizadas son como bloques de construcción que le permiten encapsular datos y funciones, haciendo que su código sea más organizado y más fácil de reutilizar.

2. ¿Por qué es importante la descomposición en la resolución de problemas, especialmente en el desarrollo de software? Elija la mejor respuesta.

- ☒ Permite descomponer un problema complejo en partes más pequeñas y manejables.
- ☐ Hace que tu código parezca más impresionante.
- ☐ Sólo es útil para proyectos muy grandes.
- ☐ Te ayuda a escribir código más rápido

✔ **Correcto**

Correcto La descomposición es el proceso de dividir un problema grande en partes más pequeñas y fáciles de manejar, haciéndolo más abordable y solucionable.

3. Está creando una función para que el sistema de punto de venta de un restaurante calcule la factura total de un cliente. El coste base de la comida viene determinado por los artículos pedidos. Sin embargo, el restaurante ofrece descuentos a determinados grupos de clientes:

- Miembros fieles: Los clientes que son miembros del programa de fidelidad reciben un 10% de descuento.
- Especial para madrugadores: Los clientes que cenar antes de las 17:00 reciben un descuento del 15%.

¿Cuál sería la mejor manera de estructurar su función para incorporar estos descuentos de forma eficaz? Seleccione el mejor enfoque. Seleccione la mejor respuesta.

- ☒ Definir parámetros opcionales para 'loyalty_member' y 'time' (para early bird) con valores por defecto para manejar diferentes escenarios de descuento
- ☐ Almacenar toda la información relacionada con el cliente y el tiempo en variables globales y hacer que la función acceda directamente a estas variables en lugar de utilizar parámetros
- ☐ Calcular los descuentos externamente y pasar sólo el importe final de la factura a la función
- ☐ Evite utilizar una función y calcule la factura total y los descuentos directamente en el flujo principal del programa

☒ **Correcto**

Correcto Este enfoque permite flexibilidad y fácil expansión si cambian las categorías de descuento.

4. ¿Cómo contribuyen las funciones a la reutilización y mantenimiento del código? Seleccione todo lo que corresponda.

☒ Fomentan la colaboración al permitir que distintos desarrolladores trabajen en funciones separadas de forma independiente.

☒ **Correcto**

Correcto Las funciones permiten un desarrollo modular, en el que los distintos miembros del equipo pueden centrarse en partes específicas del código base sin interferir en el trabajo de los demás.

☒ Facilitan las pruebas y la depuración del código al aislar unidades específicas de funcionalidad.

☒ **Correcto**

Correcto Puede probar y depurar funciones individuales, lo que simplifica la identificación y solución de problemas.

☒ Optimizan automáticamente su código para mejorar el rendimiento.

☒ **Esto no debería estar seleccionado**

No del todo. Aunque las funciones pueden contribuir a un código bien estructurado, no optimizan intrínsecamente su rendimiento.

☒ Permiten encapsular una tarea específica, facilitando la reutilización de esa funcionalidad en todo el código.

☒ **Correcto**

Correcto Las funciones actúan como bloques de construcción reutilizables, evitando que tengas que escribir el mismo código varias veces.

5. Está desarrollando una función para generar informes de usuario. Estos informes pueden incluir varias secciones como "Información personal", "Historial de pedidos" y "Preferencias" Algunos usuarios sólo necesitan un informe básico con "Información personal", mientras que otros necesitan un informe completo con todas las secciones. ¿Cómo diseñaría la función para gestionar eficazmente estos requisitos variables? Seleccione la mejor respuesta.

- ☐ Genere el informe completo y deje que el usuario elimine manualmente las secciones que no necesite.
- ☒ Definir parámetros opcionales para cada sección del informe, permitiendo al usuario especificar qué secciones necesita.
- ☐ Crear funciones separadas para cada tipo de informe (básico, detallado).
- ☐ Incluya todas las secciones posibles en la función y utilice sentencias condicionales para controlar qué secciones se incluyen en la salida.

☒ **Correcto**

Correcto Este enfoque proporciona flexibilidad y mantiene la función concisa y fácil de entender.

Módulos integrados en Python

Los **módulos integrados** son bibliotecas de código ya incluidas en Python que contienen funciones, clases y variables listas para usar. Permiten resolver problemas comunes sin tener que programar todo desde cero.

Ventajas principales

1. **Eficiencia y productividad:** ahorran tiempo al reutilizar funciones preescritas.
2. **Legibilidad y mantenimiento:** nombres descriptivos facilitan la comprensión del código.
3. **Confiabilidad y optimización:** probados y actualizados por la comunidad.
4. **Evolución constante:** se adaptan a las mejores prácticas y nuevas necesidades.

Módulos integrados más comunes

- **math:** funciones matemáticas avanzadas (ej. `sqrt`, `log`, `pi`).
 - Ejemplo: análisis financiero, trigonometría, cálculos científicos.
- **random:** generación de números y secuencias aleatorias.
 - Ejemplo: simulaciones, juegos, terrenos aleatorios.
- **datetime:** manejo de fechas y tiempos.
 - Ejemplo: planificación de tareas, informes con plazos, registro de eventos.

Otros módulos útiles

- **os:** interactúa con el sistema operativo (archivos, directorios, variables de entorno).
- **sys:** información sobre el intérprete de Python y el sistema.
- **json:** codificar y decodificar datos en formato JSON.

- **csv**: leer y escribir archivos separados por comas (CSV).

Conclusión

Los módulos integrados de Python son **herramientas poderosas y confiables** que aumentan la productividad, mejoran la claridad del código y facilitan el mantenimiento. Aprovecharlos te permite enfocarte en la parte creativa y compleja de tus proyectos.

Pregunta

¿Cuál de los siguientes escenarios demuestra un caso de uso típico para aprovechar los módulos incorporados de Python para mejorar la eficiencia y agilizar el desarrollo? Seleccione todo lo que corresponda.

- ☒ Un desarrollador web necesita implementar un sistema seguro de inicio de sesión de usuario con hashing de contraseña y autenticación.

☒ **Correcto**

Correcto Este es un caso de uso válido. Aunque no se menciona explícitamente en el script, los módulos relacionados con la seguridad como hashlib están incorporados y son esenciales para la autenticación segura de usuarios.

- ☒ Un científico de datos necesita visualizar los resultados de sus análisis mediante tablas y gráficos.

☒ **Correcto**

Correcto Este es también un caso de uso común. Bibliotecas como Matplotlib, a menudo consideradas módulos integrados, ofrecen potentes herramientas para la visualización de datos.

- ☒ Un desarrollador de juegos quiere introducir un elemento de aleatoriedad e imprevisibilidad en la mecánica de su juego.

☒ **Correcto**

Correcto Este es un caso de uso válido. El módulo random proporciona funciones para generar números aleatorios y secuencias, por lo que es adecuado para introducir aleatoriedad en los juegos.

¿Qué son las bibliotecas externas?

Son **conjuntos de herramientas creados por la comunidad de Python** que amplían las capacidades del lenguaje más allá de sus módulos integrados. Permiten realizar tareas complejas de forma más fácil y eficiente (ej. visualización de datos, desarrollo web, aprendizaje automático).

Beneficios principales

1. **Eficiencia y productividad**: soluciones listas para usar sin reinventar la rueda.
2. **Legibilidad y mantenimiento**: funciones con nombres descriptivos → código más claro.

3. **Confiabilidad:** desarrolladas y optimizadas por expertos en cada campo.
4. **Actualizaciones constantes:** mantenidas por la comunidad global de Python.
5. **Especialización:** diseñadas para dominios específicos como IA, visualización, análisis de datos, etc.

Ejemplos de bibliotecas externas

- **Desarrollo web:**
 - *Flask, Django:* crear aplicaciones web escalables.
 - *Requests:* trabajar fácilmente con APIs y peticiones HTTP.
- **Ciencia de datos:**
 - *NumPy:* cálculos numéricos y matrices.
 - *Pandas:* análisis y manipulación de datos.
 - *Matplotlib:* visualización de datos en gráficos y tablas.
- **Aprendizaje automático e IA:**
 - *Scikit-learn:* clasificación, regresión, clustering, etc.
 - *PyTorch, Keras, Microsoft Cognitive Toolkit:* redes neuronales y deep learning.

Instalación con pip

1. Abrir la terminal o línea de comandos.

Escribir:

```
pip install library_name
```

2. (reemplazar *library_name* por el nombre de la librería).

3. Pip descarga e instala la biblioteca junto con sus dependencias.

Conclusión

Las **bibliotecas externas** son la clave para aprovechar al máximo Python:

- Te ahorran tiempo.
- Hacen el código más claro y mantenible.
- Te permiten acceder a herramientas avanzadas para proyectos de datos, IA, web y más.

Pregunta

¿Cuál de las siguientes afirmaciones refleja con exactitud el papel de las bibliotecas externas en la programación en Python?

- ☐ Las bibliotecas externas son esenciales para las funciones básicas de Python, como imprimir resultados o realizar operaciones aritméticas.
- ☒ Las bibliotecas externas ofrecen funcionalidades especializadas que van más allá de las capacidades básicas de Python, lo que permite a los desarrolladores abordar tareas complejas con eficacia.
- ☐ Las bibliotecas externas se utilizan principalmente para crear estructuras de datos y algoritmos personalizados a partir de cero.
- ☐ Las bibliotecas externas son desarrolladas y mantenidas principalmente por grandes empresas, lo que limita las aportaciones de la comunidad y la innovación.

✓ **Correcto**

Correcto Captura con precisión la esencia de las bibliotecas externas como herramientas que amplían la funcionalidad de Python, proporcionando soluciones listas para usar y acelerando el desarrollo.

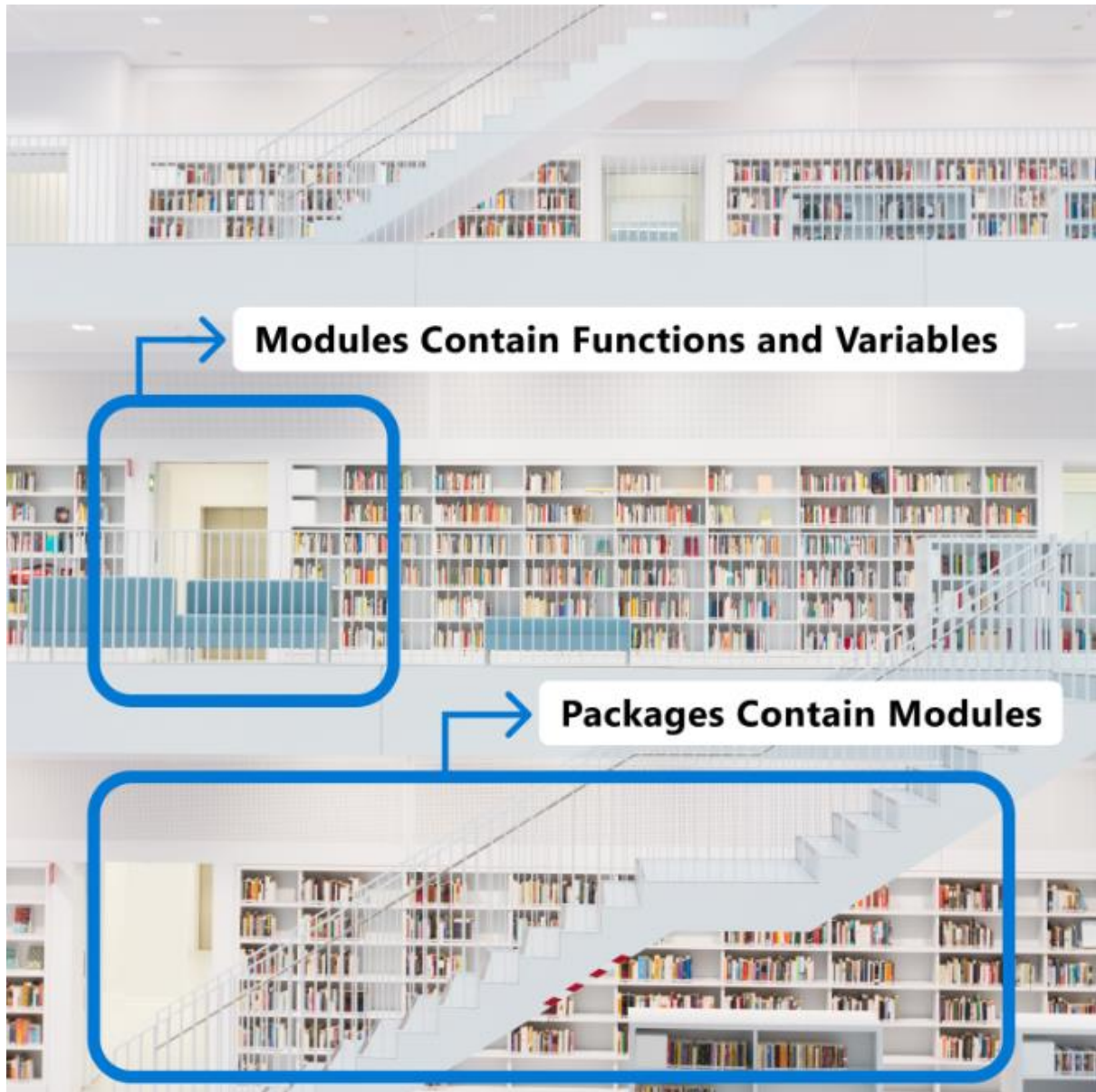
Gestión de paquetes con pip: Instalación y actualización de bibliotecas

Python, pip y Conda

Módulos, paquetes y bibliotecas

- **Módulo** → archivo con código Python.

- **Paquete** → colección de módulos relacionados.
- **Biblioteca** → conjunto más amplio de paquetes con herramientas completas para un dominio (ej. *pandas* para análisis de datos, *Django* para desarrollo web).



□ ¿Qué es pip?

- Es el instalador de paquetes por defecto en Python.

- Permite **instalar, actualizar y gestionar** bibliotecas desde **PyPI** (Python Package Index).

Se usa con comandos en la terminal:

```
pip install nombre_paquete
```

```
pip install nombre_paquete==versión
```

```
pip update nombre_paquete
```

```
pip uninstall nombre_paquete
```

```
pip list
```

-
- Facilita trabajar con proyectos de análisis de datos, web, IA, etc.

¿Qué es Conda?

- Es un **gestor de paquetes y entornos multiplataforma**.
- Puede manejar dependencias de **Python y otros lenguajes** (R, Java, C, etc.).
- Permite crear **entornos aislados**, evitando conflictos entre proyectos.

Comandos clave de Conda

- Crear un entorno: `conda create -n mi_entorno`
- Activar / desactivar: `conda activate mi_entorno` | `conda deactivate`
- Instalar: `conda install paquete`
- Actualizar: `conda update paquete`
- Eliminar: `conda remove paquete`
- Ver entornos: `conda env list`

Conda vs. pip

- **pip**: ideal para proyectos Python puros, rápido y directo desde PyPI.
- **Conda**: mejor para proyectos **multilingües** o con **dependencias complejas** (ej. ciencia de datos con librerías que requieren librerías de C, R, etc.).

Idea principal

- **pip** te da acceso al enorme ecosistema de PyPI y es la herramienta estándar de Python.
- **Conda** va más allá, gestionando entornos completos y múltiples lenguajes.
- Usar uno u otro depende del proyecto:
 - Proyectos ligeros → **pip**.
 - Proyectos grandes, con dependencias de varios lenguajes → **Conda**.

Gestión de paquetes con pip

¿Por qué usar pip?

- Instalar paquetes manualmente es tedioso y propenso a errores.
- **pip** automatiza la instalación, gestión de dependencias, control de versiones y compatibilidad.
- Permite instalar con un solo comando desde **PyPI** (Python Package Index).

Instalación de paquetes

Ejemplo:

```
pip install django
```

-

- pip descarga Django y todas sus dependencias automáticamente.

Puedes instalar **varios paquetes a la vez**:

```
pip install django djangorestframework django-cors-headers
```

-

Actualización y control de versiones

Actualizar un paquete:

```
pip install --upgrade django
```

-

Instalar una versión específica:

```
pip install django==1.23.5
```

-

Archivos de requisitos (requirements.txt)

- Guardan la lista de dependencias con versiones exactas.

Reproducen entornos en otros equipos con:

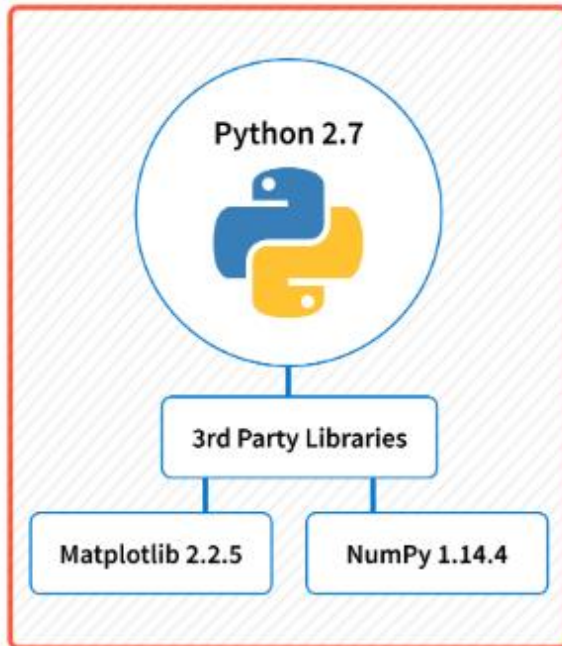
```
pip install -r requirements.txt
```

-

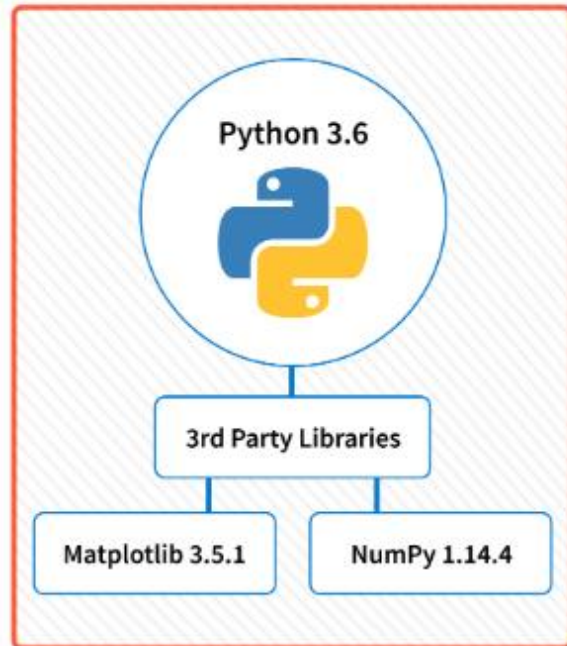
Entornos virtuales

- Crean “**cajas de arena**” aisladas para cada proyecto.
- Evitan conflictos de versiones entre proyectos.
- Cada entorno tiene sus propios paquetes independientes.

Virtual Environment #1



Virtual Environment #2



Buscar paquetes en PyPI

- Se recomienda usar pypi.org para encontrar la librería adecuada.

Comandos útiles de pip

- `pip list` → muestra paquetes instalados.
- `pip show paquete` → info detallada de un paquete.
- `pip freeze` → genera lista para requirements.txt.
- `pip check` → detecta conflictos de dependencias.

Idea principal

Dominar **pip** es esencial para cualquier desarrollador Python:

- Permite instalar, actualizar y gestionar bibliotecas de forma rápida.
- Garantiza entornos reproducibles y colaborativos.
- Junto con entornos virtuales, asegura un flujo de trabajo limpio y sin conflictos.

Bibliotecas Python y el poder de la comunidad

La fuerza de Python no solo está en su sintaxis sencilla, sino también en su **gran comunidad** que desarrolla **bibliotecas externas**. Estas librerías ofrecen código preescrito para tareas específicas, lo que evita “reinventar la rueda” y acelera el desarrollo.

Principales bibliotecas para Ciencia de Datos y Machine Learning

1. NumPy

- Base de la computación numérica en Python.
- Manejo eficiente de **arrays y matrices** (almacenados de forma contigua en memoria).
- Permite operaciones matemáticas **rápidas y vectorizadas** sin bucles explícitos.
- Ejemplo: sumar dos listas grandes en una sola operación.

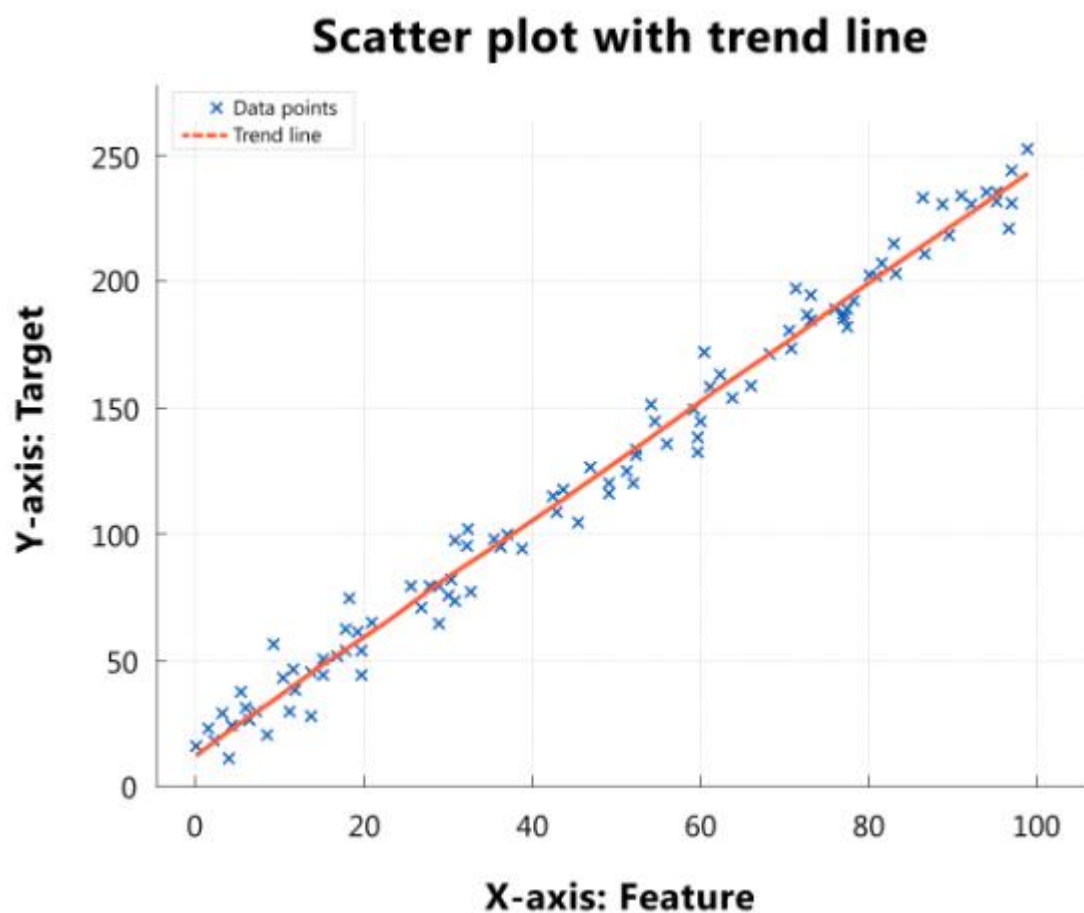
2. pandas

- Construido sobre NumPy, especializado en **análisis y manipulación de datos**.
- Usa la estructura **DataFrame** (similar a una tabla de Excel o SQL).
- Facilita:
 - Limpieza y transformación de datos.
 - Manejo de datos faltantes.

- Análisis de series temporales.
- Tablas dinámicas y estadísticas resumidas.
- Ejemplo: leer un CSV de ventas, filtrar por región, calcular totales por producto y hacer tablas cruzadas.

3. Matplotlib

- Biblioteca de **visualización flexible y personalizable**.
- Permite controlar colores, estilos, etiquetas y ejes.
- Soporta gráficos **interactivos** (zoom, tooltips, leyendas dinámicas).
- Ejemplo: gráfico de dispersión con puntos codificados por color, línea de tendencia y leyenda interactiva.



Idea principal

Gracias a su comunidad, Python cuenta con bibliotecas como **NumPy**, **pandas** y **Matplotlib**, que forman la **trilogía esencial de la ciencia de datos**:

- Cálculo numérico rápido.
- Manipulación y análisis de datos.
- Visualización clara y personalizable.

4.

Scikit-learn: Librería para **Machine Learning tradicional**. Incluye algoritmos de clasificación, regresión y clustering, además de herramientas para **preprocesamiento, validación cruzada y evaluación de modelos**.

- Ejemplo: clasificar opiniones de clientes como positivas o negativas.

5.

Microsoft Cognitive Toolkit (CNTK) y PyTorch: Marcos de **deep learning**.

- CNTK: rápido y eficiente en grandes redes neuronales.
- PyTorch: flexible gracias a su **gráfico dinámico de cálculos**, ideal para investigación.
- Ejemplo: usar redes neuronales convolucionales (CNN) para clasificar imágenes.

Desarrollo web

- **Django:** Framework completo, con filosofía *“baterías incluidas”*. Ofrece ORM, motor de plantillas, administración integrada y seguridad.
 - Ejemplo: construir un blog con autenticación, comentarios y búsqueda.

Starting Flask app...

Example Admin Home User			
List (2)	Create	With selected▼	
☐		Name	Email
☐	 	Alice	alice@example.com
☐	 	Bob	bob@example.com

- **Flask:** Microframework ligero y flexible, ideal para APIs o proyectos pequeños.
 - Ejemplo: crear una API RESTful para datos meteorológicos.
- **Requests:** Facilita enviar **peticiones HTTP** y trabajar con APIs.
 - Ejemplo: obtener titulares de una API de noticias y procesar la respuesta JSON.

Extracción y procesamiento de datos

- **Beautiful Soup:** Especializada en el **web scraping** de HTML y XML, útil para extraer información estructurada.
 - Ejemplo: extraer precios y reseñas de productos en un e-commerce.
- **Scrapy:** Framework potente para **rastreo y extracción masiva de datos** de sitios web. Soporta scraping asíncrono y almacenamiento automático.
 - Ejemplo: recopilar listados inmobiliarios de múltiples páginas.

Visión por computador

- **OpenCV:** Librería líder para procesamiento de imágenes y videos. Permite detección de objetos, reconocimiento facial y análisis en tiempo real.
 - Ejemplo: rastrear personas en grabaciones de cámaras de seguridad.

Idea principal

Estas bibliotecas muestran cómo la **comunidad de Python** amplía el lenguaje hacia múltiples áreas:

- **IA y Machine Learning** → Scikit-learn, PyTorch, CNTK.
- **Desarrollo web** → Django, Flask, Requests.
- **Extracción de datos** → BeautifulSoup, Scrapy.
- **Visión por computador** → OpenCV.

Gracias a ellas, Python es un ecosistema versátil para proyectos de todo tipo.

Introducción a las bibliotecas de Python

1. Instalación y uso

- Se instalan con `pip install nombre_libreria`.
- Ejemplo: `pip install matplotlib`.
- Se utilizan importándolas en el código (`import numpy as np, import pandas as pd`).

2. La ventaja de la comunidad

- Python es de **código abierto** y tiene una **comunidad activa y colaborativa**.
- Esta comunidad mantiene, mejora y documenta las bibliotecas.
- Permite acceso a foros, tutoriales y ayuda de expertos.
- Las bibliotecas evolucionan rápido con nuevas funciones, optimizaciones y correcciones de errores.

3. Bibliotecas como herramientas de aprendizaje

- No deben verse como una “muleta”, sino como un **trampolín** para aprender y desarrollar más rápido.

- Permiten enfocarse en problemas de alto nivel sin perderse en detalles técnicos.
- Revisar el código fuente de una biblioteca ayuda a entender algoritmos y buenas prácticas.
- Muchas incluyen documentación y ejemplos diseñados con fines educativos.

4. Idea central

- Las bibliotecas de Python son **claves para ampliar las capacidades** del lenguaje.
- Representan la experiencia colectiva de la comunidad y permiten abordar tareas de datos, web, IA y más con eficiencia y elegancia.
- Usarlas no solo mejora tu productividad, sino que también es una oportunidad para **aprender de los expertos**.

Importar módulos integrados en Python

1. Qué son los módulos integrados

- Son colecciones de **funciones, clases y variables** preescritas para tareas específicas (matemáticas, fechas, sistema operativo, aleatoriedad, etc.).
- Funcionan como **conjuntos de herramientas especializados**.

2. Por qué importarlos

- **Eficiencia:** código optimizado y listo para usar.
- **Organización y legibilidad:** agrupan funciones relacionadas con nombres claros.
- **Evitan conflictos:** cada módulo tiene su propio espacio de nombres.
- **Flexibilidad:** se pueden importar módulos completos o funciones específicas.
- **Confiabilidad:** están creados y probados por la comunidad Python.

3. Cuándo usarlos

- Cuando el proyecto necesita **funcionalidad avanzada** más allá de Python básico (fechas, cálculos matemáticos, interacción con el sistema, etc.).
- En entornos con recursos limitados, se pueden importar solo las funciones necesarias.
- Para **evitar conflictos de nombres** al trabajar con funciones que se solapan con las tuyas.

4. Cómo importarlos

Módulo completo:

```
import math  
  
math.sqrt(25) # raíz cuadrada
```

○

Función específica:

```
from random import randint  
  
randint(1, 10) # número aleatorio entre 1 y 10
```

○

5. Idea central

- Importar módulos integrados te permite escribir código más eficiente, limpio y potente.
- Es una habilidad clave para expandir tus posibilidades en Python.

Pregunta

Estás trabajando en un proyecto en Python en el que necesitas calcular la raíz cuadrada de un número y generar un número aleatorio en coma flotante entre 0 y 1. ¿Cuál de las siguientes sentencias de importación sería la forma MÁS apropiada y eficiente de acceder a las funcionalidades necesarias de los módulos incorporados?

- ☐ `import math`
- ☒ `from math import sqrt; from random import random`
- ☐ `import math, random`
- ☐ `from random import *`

✔ **Correcto**

Esta es la respuesta correcta. Importa de forma selectiva sólo la función `sqrt()` del módulo `math` y la función `random()` del módulo `random`, lo que le garantiza el acceso a las funciones exactas que necesita y, al mismo tiempo, le permite concentrarse en el código.

Crear tu propio módulo en Python

1. Qué es un módulo

- Un módulo es como un *ladrillo de Lego*: un archivo que contiene **funciones**, **clases** y **variables** para tareas específicas.
- Permite construir proyectos complejos a partir de componentes simples y reutilizables.

2. Por qué son importantes los módulos

● Organización

- Evitan el caos en proyectos grandes.
- Cada módulo representa una funcionalidad distinta (ejemplo: autenticación, base de datos, interfaz web).
- Facilitan la navegación y comprensión del código.

● Reutilización

- El código puede usarse en varios proyectos sin reescribirlo.
- Ejemplo: un módulo de envío de correos puede reutilizarse en cualquier aplicación que necesite esa función.

● Gestión de espacios de nombres

- Cada módulo tiene su propio espacio de nombres.

- Evita conflictos entre funciones o variables con el mismo nombre en diferentes módulos.
- **Mantenibilidad**
 - Más fácil de actualizar y modificar sin afectar al resto del proyecto.
 - Favorece la escritura de **pruebas unitarias** por módulo, lo que ayuda a detectar errores rápidamente y mantener la calidad.

En resumen: Los módulos en Python permiten **organizar, reutilizar, proteger y mantener mejor el código**, haciendo los proyectos más claros, escalables y profesionales.

Anatomía de un módulo en Python

1. ¿Qué es un módulo?

- Un módulo en Python es **un archivo .py** que contiene código organizado.
- Puede incluir:
 - **Funciones** → bloques reutilizables para tareas específicas.
 - **Clases** → plantillas para crear objetos.
 - **Variables** → valores almacenados para usar en el código.

2. Ejemplo de módulo: **geometry_calculations.py**

```
# File: geometry_calculations.py
```

```
import math
```

```
def calculate_area_circle(radius):
```

```

    """Calcula el área de un círculo dado su radio."""
    return math.pi * radius**2

def calculate_circumference_circle(radius):
    """Calcula la circunferencia de un círculo dado su radio."""
    return 2 * math.pi * radius

class Rectangle:
    """Representa un rectángulo con ancho y alto."""

    def __init__(self, width, height):
        self.width = width
        self.height = height

    def calculate_area(self):
        """Calcula el área del rectángulo."""
        return self.width * self.height

    def calculate_perimeter(self):
        """Calcula el perímetro del rectángulo."""
        return 2 * (self.width + self.height)

```

Este módulo incluye:

- Importación de `math`.

- Dos funciones (`calculate_area_circle`, `calculate_circumference_circle`).
- Una clase `Rectangle` con métodos para área y perímetro.

3. Uso del módulo en otro archivo: `main.py`

```
# File: main.py
```

```
import geometry_calculations
```

```
radius = 5
```

```
area = geometry_calculations.calculate_area_circle(radius)
```

```
circumference =  
geometry_calculations.calculate_circumference_circle(radius)
```

```
print(f"Area of circle: {area}")
```

```
print(f"Circumference of circle: {circumference}")
```

```
rect = geometry_calculations.Rectangle(4, 6)
```

```
rect_area = rect.calculate_area()
```

```
rect_perimeter = rect.calculate_perimeter()
```

```
print(f"Area of rectangle: {rect_area}")
```

```
print(f"Perimeter of rectangle: {rect_perimeter}")
```

Aquí se importan y usan las funciones y clases del módulo.

4. Técnicas alternativas de importación

- Importar elementos específicos

```
from geometry_calculations import calculate_area_circle, Rectangle
```

- Usar un alias para el módulo

```
import geometry_calculations as geo
```

- Importar todo el contenido (no recomendado en módulos grandes)

```
from geometry_calculations import *
```

En resumen:

Un módulo en Python es una forma de organizar y reutilizar código. Puedes crearlo, importarlo en otros archivos y aprovechar distintas formas de importación según lo que necesites.

Conceptos avanzados de módulos

1. Conceptos clave

- **Paquetes:** Colecciones de módulos organizados en una estructura jerárquica.
- **Importaciones relativas:** Permiten importar módulos dentro del mismo paquete.
- **Importaciones condicionales:** Cargar módulos solo si se cumplen ciertas condiciones.
- **Recarga de módulos:** Permite actualizar un módulo ya importado para reflejar cambios.

2. Crear un módulo Python

1. Elegir un nombre significativo para el módulo: `math_utils.py`, `data_analysis_tools.py`.
2. Crear un archivo `.py` con funciones, clases y variables.
3. Agregar **docstrings** y nombres claros para mejorar la legibilidad.
4. Importar el módulo en otros archivos usando `import module_name`.

Ejemplo: `data_analysis_tools.py`

```
# File: data_analysis_tools.py
```

```
import pandas as pd
```

```
import numpy as np
```

```
def calculate_mean(data):
```

```
    """Calculates the mean of a list of numbers."""
```

```
    return np.mean(data)
```

```
# ... otras funciones para mediana, moda, desviación estándar
```

Uso en `main.py`

```
# File: main.py
```

```
import data_analysis_tools
```

```
my_data = [2, 4, 5, 8, 1, 9]

mean_value = data_analysis_tools.calculate_mean(my_data)

print(f"Mean: {mean_value}")
```

3. Organización del proyecto

- Crear un directorio principal: `my_project/`
- Archivos iniciales:

```
my_project/

    main.py

    geometry_calculations.py
```

Ejemplo de módulo: `geometry_calculations.py`

```
# File: geometry_calculations.py
```

```
import math
```

```
def calculate_area_circle(radius):

    return math.pi * radius**2
```

```
def calculate_circumference_circle(radius):

    return 2 * math.pi * radius
```

```
class Rectangle:
```

```
def __init__(self, width, height):  
    self.width = width  
    self.height = height  
  
def calculate_area(self):  
    return self.width * self.height  
  
def calculate_perimeter(self):  
    return 2 * (self.width + self.height)
```

Uso en **main.py**

```
# File: main.py  
  
import geometry_calculations  
  
radius = 5  
  
area = geometry_calculations.calculate_area_circle(radius)  
  
print(f"Area of circle: {area}")
```

- Python importa automáticamente los módulos del **mismo directorio**.
- Para proyectos más grandes, crear un subdirectorio **modules/** para alojar todos los módulos personalizados y mantener el código limpio y organizado.

Beneficios de dominar módulos

- Código más **organizado y modular**.
- Facilita la **reutilización de funciones y clases**.
- Proyectos más **mantenibles y escalables**.
- Facilita la colaboración en proyectos grandes.

(TODO LO DE CREAR TU PROPIO MÓDULO Y LO DEL LABORATORIO MIRARLO EN VS)

Cuestionario

1. Un equipo de programadores está desarrollando una aplicación financiera. ¿Cuál de las siguientes tareas se realizaría mejor utilizando el módulo matemático? Seleccione la mejor respuesta.
- ☐ Generación de números aleatorios para simular fluctuaciones bursátiles
 - ☐ Formato de fechas y horas para registros de operaciones
 - ☒ Cálculo del interés compuesto de las inversiones
 - ☐ Lectura y escritura de datos financieros desde Archivos CSV

✔ **Correcto**

Correcto El módulo `math` proporciona funciones para realizar cálculos matemáticos complejos, incluidos los necesarios para el interés compuesto.

2. Un científico de datos está trabajando en un proyecto que implica el análisis de un gran conjunto de datos con información sobre datos demográficos de los clientes, historial de compras y preferencias de productos. ¿Qué bibliotecas externas serían más útiles para este proyecto? Seleccione todas las que corresponda.

☒ Scikit-learn

☒ **Correcto**

Correcto Scikit-learn proporciona herramientas de aprendizaje automático que podrían utilizarse para construir modelos predictivos basados en los datos de los clientes.

☐ Flask

☒ pandas

☒ **Correcto**

Pandas está diseñado específicamente para la manipulación de datos y el análisis, por lo que es una herramienta fundamental para este proyecto.

☒ NumPy

☒ **Correcto**

Correcto NumPy es excelente para la computación numérica y sería útil para manejar los aspectos numéricos del conjunto de datos.

3. Estás comenzando un nuevo proyecto que requiere varias librerías externas. ¿Cuál de los siguientes comandos usarías para instalar la librería 'requests', que se usa para hacer peticiones HTTP, y la librería 'beautifulsoup4', que se usa para web scraping? Selecciona la mejor respuesta.

☐ `install requests and beautifulsoup4 with pip`

☒ `pip install requests beautifulsoup4`

☐ `conda install -r requirements.txt`

☐ `pip requests beautifulsoup4 install`

☒ **Correcto**

Correcto Este comando utiliza correctamente pip para instalar ambas librerías en una sola línea.

4. Un desarrollador está creando una aplicación web que necesita interactuar con una API meteorológica para obtener y mostrar datos meteorológicos en tiempo real. ¿Cuál de las siguientes bibliotecas de Python sería más esencial para esta tarea? Selecciona la mejor respuesta.

- ☒ Solicita
☐ Django
☐ pandas
☐ Matplotlib

Request



Correcto La librería Requests está específicamente diseñada para realizar peticiones HTTP, lo cual es crucial para interactuar con APIs.

5. Estás desarrollando un juego que requiere el uso de números aleatorios y cálculos trigonométricos. ¿Qué módulos necesitarías importar para cumplir estos requisitos? Selecciona todos los que correspondan.

☐ `datetime`

☒ `random`



Correcto El módulo `random` proporciona funciones para generar números aleatorios.

☐ `os`

☒ `math`



Correcto El módulo `math` proporciona funciones para cálculos trigonométricos.

6. Un equipo de desarrolladores está trabajando en un gran proyecto con numerosas funcionalidades, como la autenticación de usuarios, el tratamiento de datos y las interacciones con la API. Quieren organizar su código base de forma eficaz para mejorar la mantenibilidad y la reutilización. ¿Cuál de las siguientes estrategias se alinea mejor con el concepto de módulos para lograr este objetivo? Seleccione la mejor respuesta.

- ☐ Escriba todo el código en un único archivo con comentarios claros para delimitar las distintas funcionalidades.
- ☐ Utilice un sistema de control de versiones como Git para realizar un seguimiento de los cambios y colaborar, pero mantenga todo el código dentro de un único módulo.
- ☐ Defina todas las funciones y clases dentro de un mismo módulo, pero utilice diferentes convenciones de nomenclatura para distinguir las funcionalidades.
- ☒ Crea módulos independientes para cada funcionalidad (por ejemplo, `authentication.py`, `data_processing.py`, `api_interactions.py`) e impórtalos según sea necesario.

✓ **Correcto**

Correcto Esto ejemplifica el enfoque modular, que promueve la organización, la reutilización y la mantenibilidad.

ACTIVIDAD FINAL

1. Imagina que estás desarrollando un programa para calcular el área de diferentes figuras. Necesitas calcular el área de un círculo, un cuadrado y un triángulo varias veces a lo largo del programa. ¿Cómo pueden ayudarte las funciones en este caso? Selecciona la mejor respuesta.

- ☒ Al permitirle escribir las fórmulas de cálculo del área una vez y reutilizarlas para cada forma
- ☐ Facilitando la localización y corrección de errores, ya que las funciones aíslan las secciones de código
- ☐ Reduciendo la cantidad de memoria que utiliza el programa porque las funciones almacenan los cálculos de forma eficiente
- ☐ Haciendo que el programa se ejecute más rápido porque las funciones ejecutan el código más rápido

✓ **Correcto**

¡Correcto! Las funciones permiten definir los cálculos una vez y reutilizarlos, reduciendo la redundancia y mejorando la organización del código.

2. En un programa de simulación diseñado para modelar un ecosistema virtual, ¿cuál de los siguientes ejemplos demuestra el concepto de polimorfismo? Seleccione la mejor respuesta.

- ☐ Una clase "Planta" con un método "crecer" que aumenta el tamaño de la planta.
- ☐ Una clase "Depredador" que hereda de una clase "Animal".
- ☒ Un método "mover" que se implementa de forma diferente para una clase "Pájaro" (volar) y una clase "Pez" (nadar).
- ☐ Una clase "Animal" con atributos como "edad" y "peso"

✓ **Correcto**

Correcto Esto ilustra el polimorfismo, donde diferentes clases responden a la misma llamada de método ("mover") en su propia manera única.

3. Está desarrollando un programa para procesar grandes archivos de texto que contienen reseñas de clientes. Necesita realizar varias operaciones en cada reseña, como eliminar puntuación, convertir a minúsculas e identificar palabras clave. ¿Cuál de las siguientes opciones demuestra mejor las ventajas de utilizar funciones en este caso? Seleccione todas las que correspondan.

- ☐ Mayor velocidad de ejecución al evitar la sobrecarga de las llamadas a funciones.
- ☒ Mejora de la legibilidad del código al dividir el procesamiento complejo en funciones más pequeñas y con nombre.

✔ **Correcto**

Correcto Las funciones mejoran la legibilidad organizando el código en unidades lógicas.

- ☒ Reducción de la duplicación de código encapsulando en funciones operaciones reutilizables como "eliminar puntuación".

✔ **Correcto**

Correcto Esto se alinea con el principio DRY y promueve la mantenibilidad.

- ☒ Depuración más sencilla al aislar los posibles problemas dentro de funciones específicas.

✔ **Correcto**

Correcto Las funciones facilitan la localización y corrección de errores al aislar segmentos de código.

4. Un programador está desarrollando un juego en el que los jugadores pueden ganar puntos por completar diferentes tareas. Quiere crear una función que calcule los puntos de bonificación basándose en la puntuación actual del jugador y en el nivel de dificultad de la tarea. ¿Cuál de las siguientes opciones representa mejor los argumentos y el valor de retorno de esta función? Selecciona la mejor respuesta.

- ☐ Argumentos: nombre del jugador, nombre de la tarea; Valor de retorno: puntos totales.
- ☐ Argumentos: puntos de bonificación; Valor de retorno: puntuación del jugador, nivel de dificultad.
- ☒ Argumentos: puntuación del jugador, nivel de dificultad; Valor de retorno: puntos de bonificación.
- ☐ Argumentos: ninguno; Valor de retorno: un número aleatorio de puntos de bonificación.

✔ **Correcto**

Correcto Esto identifica con precisión las entradas (puntuación, dificultad) y la salida (puntos de bonificación) de la función.

5. Un equipo de desarrollo está trabajando en un proyecto con una gran base de código. Quieren asegurarse de que sus funciones son fácilmente comprensibles y mantenibles. ¿Cuál de las siguientes convenciones de nomenclatura sería la más adecuada para una función que comprueba si un usuario está autorizado a acceder a un recurso específico? Selecciona la mejor respuesta.

- ☒ `is_authorized_to_access`
- ☐ `isUserAuthorizedToAccess`
- ☐ `checkUserAccess`
- ☐ `auth`

✔ **Correcto**

Correcto Este nombre indica claramente el propósito de la función y sigue la convención recomendada snake_case.

6. Un equipo de desarrollo de software está creando una aplicación de streaming de música. Deciden utilizar un enfoque funcional para estructurar su código. ¿Cuál de los siguientes beneficios es más probable que experimenten al adoptar esta estrategia? Selecciona la mejor respuesta.

- ☐ Menor flexibilidad y adaptabilidad a las necesidades cambiantes
- ☐ Mayor complejidad del código y menor legibilidad
- ☐ Menor reutilización del código y mayor redundancia
- ☒ Mayor capacidad para gestionar errores y excepciones

✔ **Correcto**

Correcto Las funciones pueden mejorar la gestión de errores aislando posibles problemas y facilitando su depuración.

7. Está creando una función para enviar correos electrónicos. La dirección de correo electrónico del destinatario es obligatoria, pero el asunto y el cuerpo del correo electrónico tienen valores por defecto. ¿Cuál de las siguientes opciones demuestra el uso correcto de valores predeterminados y parámetros obligatorios en este caso? Seleccione la mejor respuesta.

- ☐ `def send_email(recipient, subject, body):`
- ☒ `def send_email(recipient, subject="Important Notice", body="This is a test email"):`
- ☐ `def send_email(subject, body, recipient="user@example.com"):`
- ☐ `def send_email(subject="Important Notice", body="This is a test email", recipient):`

✓ **Correcto**

Correcto Esto define correctamente el destinatario como un parámetro obligatorio y proporciona valores predeterminados para el asunto y el cuerpo.

Explicación:

En Python, **los parámetros obligatorios deben ir antes que los opcionales**. Aquí, `recipient` es obligatorio, mientras que `subject` y `body` tienen valores por defecto. Las otras opciones son incorrectas porque:

- `def send_email(recipient, subject, body):` → todos los parámetros son obligatorios.
- `def send_email(subject, body, recipient="user@example.com"):` → el parámetro obligatorio (`recipient`) no está primero.
- `def send_email(subject="Important Notice", body="This is a test email", recipient):` → los parámetros obligatorios no pueden ir después de los opcionales; esto genera un error de sintaxis.

8. Un equipo de desarrolladores está colaborando en un proyecto de aprendizaje automático. Quieren asegurarse de que todos los miembros del equipo utilizan las mismas versiones de las bibliotecas necesarias, como TensorFlow, Keras y Scikit-learn, para evitar problemas de compatibilidad. ¿Cuál de las siguientes prácticas facilitaría mejor este objetivo? Seleccione todas las que correspondan.

☒ Creación de un archivo requirements.txt con una lista de todas las dependencias del proyecto y sus versiones.

☒ **Correcto**

Correcto Un archivo requirements.txt garantiza que todos los miembros del equipo utilicen las mismas versiones de biblioteca.

☐ Dar instrucciones a cada miembro del equipo para que instale manualmente las bibliotecas necesarias mediante pip.

☐ Confiar en la instalación por defecto de Python en todo el sistema para todas las dependencias del proyecto.

☒ Uso de entornos virtuales para aislar las dependencias del proyecto de otros proyectos Python.

☒ **Correcto**

Correcto Los entornos virtuales ayudan a mantener la coherencia y a evitar conflictos entre distintos proyectos.

9. Un Analista de datos trabaja en un proyecto de análisis estadístico y necesita calcular la media, la mediana y la moda de un conjunto de datos. También quiere asegurarse de que su código está bien estructurado y evita posibles conflictos de nombres. ¿Qué enfoques se ajustan a estos objetivos? Seleccione todos los que correspondan.

☒ Importe sólo las funciones específicas necesarias del módulo `statistics`, como `mean`, `median` y `mode`.

☒ **Correcto**

Correcto Este enfoque garantiza que el código esté centrado y evita posibles conflictos de nombres.

☒ Importe el módulo `statistics` para utilizar sus funciones para cálculos estadísticos.

☒ **Correcto**

Correcto El módulo `statistics` proporciona funciones para operaciones estadísticas comunes.

☐ Copie y pegue fragmentos de código de recursos en línea para realizar los cálculos estadísticos.

☐ Definir sus propias funciones para calcular la media, la mediana y la moda dentro del script.

10. Un desarrollador ha creado un módulo llamado `image_processing.py` que contiene funciones para tareas comunes de manipulación de imágenes como redimensionar, recortar y aplicar filtros. Quiere utilizar estas funciones en un script separado para un proyecto de edición de imágenes por lotes. ¿Cuál es la forma correcta de conseguirlo? Seleccione la mejor respuesta.

- ☐ Cambie el nombre de `image_processing.py` a `main.py` y ejecute el script de edición de imágenes por lotes en el mismo directorio.
- ☒ Importe el módulo `image_processing` en el script de edición de imágenes por lotes utilizando la sentencia `import`.
- ☐ Copie y pegue todo el código de `image_processing.py` en el script de edición de imágenes por lotes.
- ☐ Cargue `image_processing.py` en un repositorio en línea y descárguelo dentro del script de edición de imágenes por lotes.

☒ **Correcto**

Correcto Importar el módulo permite acceder a sus funciones en el script independiente.